

Third-Party Moderation of Abuse Reports for End-to-End Encrypted Messaging with Multiple Moderators

Abstract—Abuse reporting tools like message franking allow end-to-end encrypted (E2EE) messaging platforms to verify user-generated abuse reports as part of their platform content moderation policies. While the lightweight message franking protocol deployed by Meta’s Messenger is designed with the assumption that the moderator and the platform processing messages are the same entity, proposals for other message franking-style protocols allow for a separation between the platform and moderator, albeit at a higher cost. Unfortunately, all works in this space assume that there exists a *single* moderator working with a platform who verifies and moderates all reports. This results in a situation where technical limitations limit platforms’ moderation policy options.

This paper introduces new techniques that allow E2EE messaging platforms to work with *multiple* vetted moderators, giving users options in their choice of moderators for messages they send to their friends. Verifiable abuse reporting in a multi-moderator setting requires new security notions to capture deniability requirements with respect to the platform and other moderators, as well as new privacy requirements with respect to users’ choice of moderator(s). We comprehensively study these requirements and propose three protocols for verifiable abuse reporting in this setting, offering a range of security and performance tradeoffs for different deployment scenarios. We evaluate the performance of our proposed schemes, showing that in many cases they match or exceed the performance of prior schemes that only support a single moderator.

1. Introduction

In our politically charged times, platform content moderation policies have come under siege. Increasingly, platforms have turned away from powerful and centralized moderation policies in favor of user-contributed notes [2], [3]. While this allows platforms to wash their hands of formal responsibility for moderation, it also removes access to the benefits of knowledgeable and consistent moderators.

This paper deals with content moderation tools for end-to-end encrypted (E2EE) messaging. These tools have faced less public scrutiny than those for other forms of social media, but similar moderation questions appear in this space [34]. However, content moderation policies for E2EE messaging are intimately connected to the technological tools needed to enable verifiable abuse reporting. Since platforms are not able to see users’ messages when they are sent (as they are encrypted), they may rely more heavily on user reports, where a user sends the platform a purportedly

offensive message it received, along with information about the sender. The platform needs to be able to verify that reports are authentic in order to provide fair and reliable moderation services.

Unfortunately, this is a setting where the technologies currently available restrict platforms’ policy options. Facebook Messenger uses a verifiable abuse reporting technique called *message franking* [14], [18], [31]. But message franking requires the platform’s and moderator’s operations to be closely connected, including by sharing key material. This means that it is not possible for the platform to delegate moderation to a third party without also retaining the power to verify reports itself should it want or be compelled to do so. While some prior works have augmented message franking to allow for a third party to serve as moderator [21], [36], albeit at a higher performance/communication cost, much remains to be done.

This paper considers the case of verifiable abuse reporting in the setting where there are *multiple* third-party moderators affiliated with an E2EE messaging system. We propose schemes where the platform can vet a number of moderators, e.g., representing different geographic boundaries, policy positions, or opt-in community norms. Users can then select which moderators they trust to moderate their E2EE messages for each conversation they have on the platform, and whenever their messages are reported, they are sent to the appropriate moderator for handling. This approach allows platforms to only be responsible for approving reasonable moderators, while still allowing users to benefit from professional moderation that aligns with their preferences or the legal requirements of their jurisdiction. We call a protocol that supports this kind of moderation a *third-party franking* scheme.

Allowing for multiple third-party moderators introduces new security and functionality requirements not present in prior works. We undertake a thorough analysis of the relevant requirements and introduce new definitions that capture the appropriate security notions. This includes new deniability definitions, including a notion of “subpoena safety” – a requirement that the platform cannot be compelled to verify user report data. Subpoena safety prevents the platform from verifying reports, even under duress, and thus fully delegates moderation responsibilities to third parties. Since leaking a user’s choice of moderator may leak information about the user’s preferences or location, we also introduce a *moderator privacy* notion, capturing the requirement that the platform should not even know which moderator(s) are assigned to a given user’s messages.

After describing our definitional framework for multi-moderator abuse reporting, we introduce three new verifiable abuse reporting schemes in the multi-moderator setting. These schemes all have different performance and security tradeoffs, so they are suitable for a variety of deployment scenarios. The first is a particularly lightweight scheme that relies only on standard cryptographic primitives but does not offer moderator privacy. The remaining two do offer moderator privacy, with one scheme’s costs increasing linearly in the number of moderators supported by the platform and the other offering constant, albeit somewhat higher, costs. These moderator privacy constructions make use of a blinded form of proxy re-encryption [4], [5], [19], [22] to allow the platform to send abuse report information to a user-designated moderator without the platform knowing which moderator can verify that message. Similarly, our final construction uses proxy re-encryption style techniques to convert a MAC under the platform’s key into one under the designated moderator’s key.

We implement all three schemes and report on their performance compared to existing techniques for third-party moderation in the single-moderator setting, showing comparable or even lower costs than prior work despite the added support for multiple moderators. Our implementation and raw evaluation data are available at <https://repo-link-here>.

In summary, this paper makes the following contributions.

- Describes third-party franking and introduces relevant security definitions.
- Provides three new constructions of third-party franking, offering different performance and security tradeoffs.
- Implements and evaluates the three new schemes, comparing them with prior work in the single moderator setting.

2. Technical Overview

This section briefly summarizes the paper’s technical contributions, which are explored in more detail in subsequent sections. The description here makes a number of simplifications in order to streamline presentation and elide technical details that are discussed in detail in Sections 5 and 6. Looking ahead, Section 3 formalizes our schemes’ syntax and security requirements, showing exactly how it fits into the E2EE messaging flow, and Section 4 shows where our technical contributions fit into the broader context of a complete messaging application.

Notation. We denote assignment of the value x to y using $y \leftarrow x$, and assignment to y uniformly at random from $\mathcal{X}$ as $y \xleftarrow{\mathcal{R}} \mathcal{X}$. Vectors, initialized as $v \leftarrow []$, are ordered lists of values, and elements can be added to vectors via $v.append(val)$. A function $\text{negl}(x)$ is *negligible* if, for all $c > 0$, there exists an x_0 such that, for all $x > x_0$, $\text{negl}(x) < \frac{1}{x^c}$. We use $\perp$ as a special character indicating protocol failure.

We sometimes subscript keys by i . In this case, i is an integer from 0 to N , where N is the number of moderators in the system. This denotes key material of the i th moderator. When it’s clear from context which moderator we mean, we drop the i subscript. When key material is subscripted with m , it is known to the moderator in question, and when it’s subscripted with p , it is known to the platform. When key material is subscripted with m, p , it’s known to both the moderator and the platform.

Background: message franking. Message franking is the protocol used by WhatsApp and Messenger to allow for verification of abuse reports [18], [31]. We refer to this scheme as “plain franking” in this paper. In this scheme, the message sender encrypts its message m into an authenticated encryption ciphertext c_1 , as in typical encryption, but it then additionally produces a commitment c_2 to the message m , with the commitment randomness r included alongside m in the plaintext for c_1 . Before delivering a message to its recipient, the platform adds some message context ctx (e.g., sender identity information and a timestamp) and computes a MAC $\sigma \leftarrow \text{MAC.Sign}(k, (c_2, \text{ctx}))$, where k is a platform-held key used for verifying abuse reports.

A message recipient gets $c_1, c_2, \text{ctx}, \sigma$, so it can decrypt c_1 to get (m, r) and verify that c_2 in fact commits to m with randomness r . To report a message, the user sends $m, r, c_2, \sigma, \text{ctx}$ to the platform, which also serves as the moderator. The platform then (1) checks that the commitment c_2 is valid, and (2) checks that the MAC σ is valid. Once these checks are complete, the platform can be confident that m is a message that really came through the platform with context ctx , and platform moderators can take action as needed.

Our work will augment the plain franking approach to consider the case of multiple third-party moderators instead of the platform acting as a single moderator.

Security goals. The core requirements of verifiable abuse reporting are that messages must have *accountability* – no malicious user can send a message that cannot be successfully reported – and reports must be *unforgeable* – no malicious user can create a report about a message that wasn’t actually sent.

In addition to these requirements, abuse reporting schemes need to ensure *deniability*. This means that only the designated moderator for a message can verify a report on that message. We discuss deniability in great detail in Section 3 as there are many nuances to consider in our setting. In the end, we require three different forms of deniability for schemes to cover all the different potential key compromise situations, e.g., deniability even if a user, moderator, or platform’s keys are compromised.

We also introduce a strong deniability notion called *subpoena safety*, which ensures that even if a platform’s keys are surreptitiously compromised, and all data received by a client (as opposed to reports which are generated based on this data) is seized off of a user’s phone, whoever holds this information still cannot verify the messages. The purpose of this definition is to assure a platform adopting third-

party franking that it will not incur any additional risks on behalf of its users by supporting abuse reporting. Messages remain deniable, even if the platform has been subpoenaed to provide its cryptographic key material. Therefore all abuse verification power lies with the moderators, and the platform cannot usurp this power, even under duress.

The preceding definitions are variants and extensions of security notions that exist for all verifiable abuse reporting schemes, although they need strengthening or tweaking for the third-party franking setting. Supporting multiple moderators also adds new security notions that do not apply in the single moderator case. The first is *moderator binding*, which requires that a malicious receiver cannot successfully redirect messages to a moderator other than the intended one. Unlike unforgeability, where a malicious user must forge a report for a nonexistent message, moderator binding ensures that a malicious user who does have a valid message to report still cannot successfully have that message verified by a non-intended moderator.

A second new requirement for multi-moderator abuse reporting is *moderator privacy*. Motivated by the fact that a user's choice of moderator may itself leak sensitive information, this definition requires that the platform does not learn which moderator is designated for each message. We present schemes both with and without this property.

Lightweight multi-moderator abuse reporting. Our first construction, which we refer to as the basic scheme, satisfies all the requirements for third-party franking except for moderator privacy. Its strong performance makes it suitable for cases where users' moderator choices are not considered sufficiently sensitive to warrant additional protection.

In this scheme, the platform shares a MAC key $sk_{m,p,i}$ with each moderator $i \in \{1, \dots, N\}$. Message sending and processing is identical to plain message franking, except that after computing a MAC $\sigma_{pt} \leftarrow \text{MAC}.\text{Sign}(sk_{m,p,i}, (c_2, \text{ctx}))$, the platform encrypts the MAC under the public key $pk_{m,i}$ of the intended moderator.

Encrypting the MAC tag under the intended moderator's public key ensures that no other moderator is able to decrypt and verify the MAC. Moreover, once a message is processed by the platform, neither the contents of a client's device nor any key held by the platform can be used to verify the report because the MAC tag can only be accessed via the moderator's decryption key.

Although our basic scheme is quite simple, many similar or simplified variations of the scheme are not secure. Having a single MAC key for all moderators would simplify the scheme but would break unforgeability, as any moderator could forge reports to any other moderator. Having the platform sign messages instead of MAC-ing would reduce the number of keys but break deniability, as it would allow third parties to verify reports under certain key compromise scenarios. On the encryption side, pushing encryption from the platform to the client would result in the moderator receiving the exact same report, but it would violate subpoena safety because a compromised device would in fact hold information (the MAC tag) that could be verified using only

platform keys without involving a moderator.

Adding moderator privacy. We introduce two additional constructions that build on the basic scheme and add moderator privacy. The first, Mod-Priv-1, has communication and computation costs that scale linearly in the number of moderators registered with the platform, whereas the second, Mod-Priv-2, has communication and computation complexity independent of the number of moderators, although the constant costs are higher. These constructions make use of pairings and proxy re-encryption (PRE), background on which is provided in Appendix A.

The crucial challenge in modifying our basic scheme to support moderator privacy is that the platform needs to produce and encrypt a MAC tag for a specific moderator without knowing which moderator to use. To achieve this, we will draw on ideas from PRE. Unfortunately, a straightforward use of PRE is insufficient, as the platform still needs to enforce moderator binding.

In our scheme, each moderator i will generate two PRE key pairs (pk_1, sk_1) and (pk_2, sk_2) , as well as a re-encryption key $k_{1 \rightarrow 2} \leftarrow sk_2/sk_1$ for converting encryptions under pk_1 to encryptions under pk_2 . The public keys pk_1, pk_2 , and the re-encryption key $k_{1 \rightarrow 2}$, are made available to all users of the system. The moderator discards sk_1 , as it will not be used after the initial setup. As before, each moderator will share a MAC key $sk_{m,p}$ with the platform.

When a client sends a message with respect to a given moderator, the message sender creates a new ephemeral public key epk that is derived from pk_1 as $epk \leftarrow pk_1^s$, where $s \xleftarrow{\mathbb{R}} \mathbb{Z}_q$, and sends this public key to the platform. Here q is the order of the group used in the PRE scheme. The plaintext for the encrypted message c_1 (delivered to the message recipient as part of message franking) is also modified to append a re-encryption key $k_{e \rightarrow 2}$:

$$k_{e \rightarrow 2} \leftarrow k_{1 \rightarrow 2}/s = sk_2/(sk_1 \cdot s).$$

This key enables re-encrypting ciphertexts from epk to pk_2 .

Since the platform does not know which moderator is going to be assigned to each message it processes, it computes a MAC σ_i for each $sk_{m,p,i}$ it holds. Intuitively, it then encrypts each element of the vector $\vec{\sigma}$ of MACs under the key epk . The message recipient then augments the steps of the basic protocol by

- 1) conducting the check $epk^{k_{e \rightarrow 2}} = pk_2$ to verify that the re-encryption key it received from the sender is well-formed,
- 2) discarding the encrypted MACs under keys held by the irrelevant moderators, and
- 3) when it wishes to make a report, re-encrypting σ_i using $k_{e \rightarrow 2}$ to get an encryption under the designated moderator's pk_2 .

Encrypting the MACs under epk provides moderator binding because it ensures that the ciphertext can only be decrypted by the sender's intended moderator via re-encryption, even though the platform does not know who that moderator is.

Our actual scheme, described in Section 6, does not exactly follow the intuition described here. Instead of en-

crypting each element of the vector $\vec{\sigma}$, the platform encrypts a fresh random string r' which is also an input to each MAC computation. Under reasonable assumptions on the construction of the MAC, this achieves the same effect as encrypting $\vec{\sigma}$ while dramatically reducing the number of public key cryptographic operations needed.

Moderator privacy with constant costs. The cost of Mod-Priv-1 scales linearly with the number of moderators because the platform computes and communicates a MAC for each registered moderator. Our second scheme, Mod-Priv-2, provides the same security properties with costs independent of the number of moderators. It follows the same intuition as Mod-Priv-1, but instead of simply encrypting a MAC, we make use of the key-homomorphic PRF [8], [32] $H(x)^k$ and pairings to build a “proxy re-MAC” scheme that will allow us to build a type of MAC tag that is produced by the platform but can only be verified by the specified moderator.

When processing messages, the platform computes a MAC $\sigma \leftarrow H(c_2, \text{ctx})^{\text{sk}_p} \in \mathbb{G}_1$, where sk_p is a key known only to the platform. The setup protocol for each moderator results in a client processing key $\text{pk}_{\text{proc}} = g_2^{\text{sk}_{m,p}/\text{sk}_p}$ that allows for unidirectional conversion from a MAC in $\mathbb{G}_1$ under sk_p to a corresponding MAC under sk_m , the moderator’s key, in the target group $\mathbb{G}_T$ of a pairing. The client computes $\sigma' \leftarrow e(\sigma, \text{pk}_{\text{proc}})$ to convert the MAC tag into one the chosen moderator can verify using its own key sk_m . In order to enforce moderator binding, we use the same proxy re-encryption trick with randomness r' , which is incorporated as an additional input into H . See Section 6 for details.

3. Security and Functionality Goals

This section formalizes the syntax for third-party franking, describes how to use it to augment an E2EE messaging system, and introduces the various security definitions that third-party franking schemes are required to satisfy.

3.1. Syntax and Usage

We define a *third-party franking scheme* as a seven-tuple of algorithms (SetupPlatform, SetupMod, Send, Process, Read, ReportGen, Moderate) with the following syntax:

- SetupPlatform(1^λ) $\rightarrow$ ($\text{sk}_p, k_{\text{reg}}$): this function takes as input a security parameter, and returns a platform secret key sk_p , as well as a moderator registration key k_{reg} .
- SetupMod($k_{\text{reg}}, 1^\lambda$) $\rightarrow$ ($\text{sk}_{m,p}, \text{sk}_m, \text{pk}_m$): this function takes as input the moderator registration key and a security parameter, and returns $\text{sk}_{m,p}$, a secret key shared between the moderator and the platform, as well as moderator secret and public keys sk_m and pk_m .
- Send($k, m, i, \text{pk}_{m,i}$) $\rightarrow$ (c_1, c_2, ad): this function takes as input a symmetric key k (shared between the sender and receiver), a message plaintext m , and the ID i and public key $\text{pk}_{m,i}$ for the intended moderator. It returns a message ciphertext c_1 , commitment to the plaintext c_2 , and some additional data ad .

- Process($\text{sk}_p, \text{ks}, c_1, c_2, \text{ad}, \text{ctx}$) $\rightarrow$ (σ, st): this function, called by the platform, takes as input the platform’s key sk_p , a vector of moderator keys ks , the output of Send, and message context ctx . It returns a tag σ and an additional message state st .
- Read($k, \text{pks}, c_1, c_2, \sigma, \text{st}$) $\rightarrow$ (m, i, rd)/ $\perp$: this function takes as input a symmetric key k and a vector of moderator public keys pks , as well as the outputs c_1, c_2 from Send and the outputs of Process. If the message is not well-formed, it returns $\perp$. Otherwise, it returns the message m , the ID i of the intended moderator, and some associated report data rd , used to generate the report.
- ReportGen(m, rd) $\rightarrow$ report: this is called to transform the output of Read into a report verifiable by the target moderator. For performance reasons, a client only calls this function on messages it wishes to report.
- Moderate($\text{sk}_m, \text{sk}_{m,p}, m, \text{report}$) $\rightarrow$ $\text{ctx}/\perp$: this function takes as input the moderator’s secret key sk_m , a secret key $\text{sk}_{m,p}$ shared with the platform, a plaintext message m , and its corresponding report. If the report accepts, it outputs the message context ctx . Otherwise, it outputs $\perp$.

All three of our schemes satisfy this same syntax, although not all inputs and outputs are used for all schemes. For example, in our first scheme the platform does not have any key material corresponding to sk_p and k_{reg} . In these cases, the unused key material is assigned the null value ε .

Note that we include two separate functions, Read and ReportGen, to capture the operations a device must do in order to read a message and prepare a report for a moderator to verify. While prior works generally use a single function to capture these two operations, we separate them out for the purposes of better describing what needs to happen every time a client receives a message, versus what happens only when a client intends to report a message.

Figure 1 shows the usage of a third-party franking scheme in the context of an E2EE messaging platform working with third-party moderators.

3.2. Security Definitions

We now describe each of the security properties required of a third-party franking scheme in more detail. Formal definitions for each security property appear in Appendix B.

Confidentiality. A straightforward requirement of any scheme built on top of E2EE messaging is that additions to the messaging protocol should not compromise the confidentiality of the underlying messaging. Since the confidentiality of all our schemes follows trivially from the security of the symmetric encryption scheme used and the hiding property of commitments, just like in standard message franking [18], [31], we do not separately define and analyze confidentiality here.

Accountability. Accountability requires all messages which are sent through the platform and accepted by the receiver to be verifiable by the target moderator. Schemes satisfying

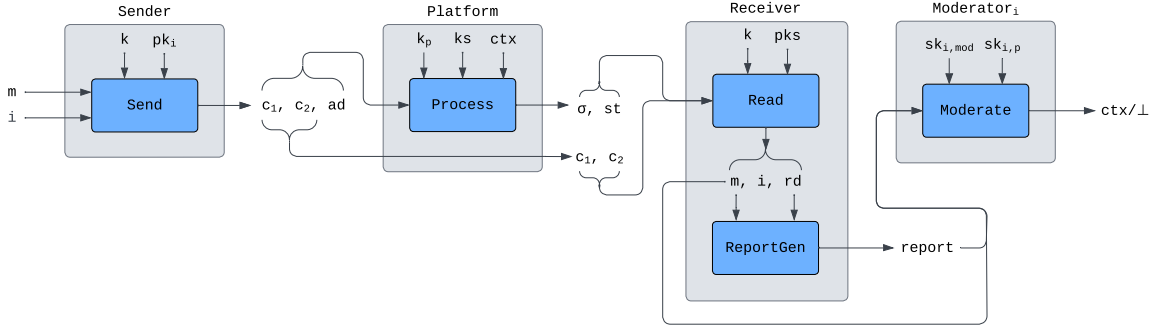


Figure 1: Illustration of how to use third-party franking in the context of E2EE messaging.

accountability protect against malicious senders who wish to send abusive messages that cannot be verifiably reported. Our accountability definition models this by allowing a malicious sender to send a chosen (c_1, c_2, ad) to an honest platform and an honest receiver. If the receiver accepts the message, i.e. Read doesn't output $\perp$, but Moderate either outputs $\perp$ or some ctx' different from the ctx assigned by the platform, the adversary wins.

As in prior work [15], [21], our definition only considers the case where the platform is doing its job honestly and a message sender is malicious, not the case where a malicious sender and malicious platform collude to deviate from the protocol. While definitions of accountability for a malicious platform have been proposed and achieved [17], [36], this usually comes at the cost of much worse performance.

Unforgeability. Unforgeability prevents a third-party franking scheme from becoming a new vector for abuse. We require that malicious users be unable to forge reports to a target moderator of their choosing, even if they have access to all other moderators' key material. Our unforgeability experiment allows the adversary to choose messages sent through the system and see the transcript of protocol messages as they are sent, processed, received, and reported to the target moderator. The adversary can also submit forgery queries $(m^*, report^*)$, and it wins if Moderate successfully outputs a ctx^* such that (m^*, ctx^*) was not previously seen.

Deniability. Intuitively, deniability definitions ensure that messages sent through the system are ephemeral, just like real-world communications, and that no one can provide "receipts" proving that a certain person sent a particular message – unless of course, they're supplying this proof to the chosen moderator.

Deniability definitions model various key compromise scenarios, and take the following form: a distinguisher is given access to some keys which have been leaked – a subset of the symmetric key of the communicating parties, the messaging platform's keys, and the moderators' keys – and it is given a report that is either an authentic one or a forgery produced by a forgery algorithm given access to another subset of these keys. A scheme has deniability if there exists

such a forgery algorithm such that any efficient adversary is unable to distinguish forged reports from genuine reports.

Since deniability definitions are parameterized by the key material available to both the distinguisher and the forger, there's a large space of possible deniability definitions. In our case, there are 64 possible definitions: 8 possible subsets of keys available to the forger, and 8 possible subsets available to the distinguisher. Some of these are not compatible with our other security goals. For example, If a forger with access to no keys was able to generate forgeries that the target moderator would be unable to distinguish from real reports, this would break our unforgeability definition.

In order to simplify our deniability analysis, we considered all deniability definitions that are compatible with our other security properties, and found three definitions that imply all others. We identified these by looking at maximally powerful distinguishers and minimally powerful forgers. For example, a forger with access to the platform's keys that can fool a distinguisher which has the target moderator's keys, will also be able to fool a distinguisher with no keys at all. Similarly, given the same forger, a distinguisher with access to no keys can do no better than a distinguisher with the target moderator's keys. Our three deniability definitions are as follows:

- **Universal deniability:** The forger has no keys, and the distinguisher has the platform keys.
- **Moderator-compromise deniability:** The forger has the target moderator's keys, and the distinguisher has the platform keys and the target moderator's keys.
- **Platform-compromise deniability:** The forger has the platform's keys, and the distinguisher has the platform keys and the target moderator's keys.

Subpoena safety. As described above, deniability is parameterized by which keys are given to which party. But our deniability definitions are only concerned with deniability of reports generated by the receiver. Subpoena safety ensures that all *inputs* to the message receiver are also deniable, and by extension, all functions of these message inputs that the receiver can compute are also deniable. Moreover, we model an attack where all keys except those of the target

moderator are compromised (perhaps via a subpoena), and the key material and data stored on a user's device are also compromised. This allows the platform a large degree of separation from moderation duties: even if the platform is compelled to attempt verification of a given message, it is unable to do so without cooperation from the message sender's chosen moderator.

We model subpoena safety in a manner similar to deniability. The main difference is in what the forgery algorithm produces: instead of a report, the forger must produce all inputs to Read.

Moderator privacy. In a setting where many moderators register with one platform, a user's choice of moderator could itself reveal information to the platform that a user may want to keep private. Our moderator privacy definition models a malicious platform, which attempts to discern which moderator is used for a given ciphertext. The adversary can send messages with a chosen, fixed moderator, in which case it sees the entire protocol transcript up to message moderation. It can also send messages that are either moderated by moderator i_0 or moderator i_1 . In this case, it sees the output of Send received by the platform and must determine which moderator was actually assigned.

Moderator binding. Moderator binding captures the property that malicious users should be unable to "redirect" a message report to a moderator that the sender has not chosen. The adversary chooses a target moderator to which it wants to redirect messages, and then it can choose messages to be sent by an honest sender with assigned moderators of the adversary's choosing. The adversary receives these messages from the platform and attempts to generate a report that will be verified by the target moderator. The adversary wins if the target moderator accepts a report that was intended for another moderator.

4. Deployment Considerations

Messaging systems are complex pieces of software with many interrelated components. To better situate our proposed algorithms in the broader context of a full messaging system, we now briefly discuss a number of deployment considerations that are crucially interrelated with abuse reporting mechanism adopted by a messaging platform.

4.1. Governance and Policy Enforcement

Third-party franking can be used to create a privacy-preserving abuse reporting mechanism, in which a moderator can be convinced that a certain client sent a particular message. The moderator can then review the message contents and take action in accordance with its policies. Defining exactly what actions a moderator should take from a policy standpoint is out of scope for our work, but we briefly describe two mechanisms by which moderators can enforce their policies.

The simplest way for moderators to take action in response to verified abuse reports is for the platform to

fully delegate trust for moderation to the moderators. In the process of vetting and registering a moderator, the platform must be convinced that the given moderator will take action against users honestly. This allows moderators to simply report their moderation decisions to the platform, which will faithfully enact their policies. This style of deployment has the advantage of being extremely simple to implement. But because each moderator has the power to globally deplatform users, this is at the cost of setting a higher bar for trust in platform moderators.

Alternatively, after registering with the platform, each moderator could act as a certificate authority. In this model, each user selects which set of moderators they find acceptable, and requests a certificate from each such moderator. The certificate represents permission to communicate in that moderator's "network." In each message sent with a given moderator, the client will also encrypt their certificate to communicate in that network, which is validated on the receiver's end. If the moderator does need to take action against a user, it can revoke the credential, essentially banning the sender from their network. The moderator could also use short-lived certificates to get around complications with certificate revocation. Such a deployment would require more overhead, but it would allow the platform to register more moderators, with significantly relaxed assumptions on moderator honesty.

4.2. Platform Honesty

It is important to note that, in the third-party moderation setting, the platform has several incentives to offload moderation responsibility onto third parties. Platforms may not have the resources to moderate effectively at scale, and thus third-party moderators can help in lightening the load for the platform. In addition, different jurisdictions can have complex and different legal requirements for the platform to enforce. Instead of accepting the legal implications of the responsibility for content moderation, the platform can delegate this task to moderators who are better able to focus on the legal requirements in their particular jurisdiction. In fact, even if the platform wanted to verify abuse reports (or was compelled to by a court subpoena), our subpoena safety property makes this impossible for the three schemes proposed here. This allows the platform to fully delegate moderation responsibilities to third party moderators.

Note that our work does not address questions of "strong accountability." That is, if a malicious platform colludes with a malicious user, the platform can render all messages from that user unreportable by tampering with the franking information it adds to messages. There exist known techniques to address this threat model, including trap messages and zero-knowledge proofs of honesty [17]. These can be directly layered with the schemes proposed here, and so we do not explore this direction further in this paper.

4.3. Integration with Existing E2EE Systems

Like all works in this space, our work focuses on the operation of the abuse reporting mechanism and abstracts away many other aspects of private messaging. For example, our Send and Read functions assume that clients communicating with each other share a key k . We make no assumptions about how they arrive at this key, e.g., it could be a single pre-shared key or a key produced by the double ratchet protocol [33]. Like prior work in this space [18], [21], [36], we use the underlying E2EE messaging protocol as a black box. Depending on the implementation, the E2EE protocol could provide security properties such as forward and backward secrecy.

We also do not specify how users get each others' public keys or the public keys of the moderators. Depending on how this is implemented, it could have ramifications for the end-to-end security properties of the resulting messaging system, so the best choice is to use an appropriate key transparency mechanism [10], [11], [20], [25], [26], [27], [29], [35], [37]. The process by which a platform selects and vets moderators, while crucial to the effectiveness of a third-party franking scheme, is not in the scope of our technical contributions.

Finally, we do not specify the protocol by which users come to agree on which moderator to use for each message. Of course the sender and receiver of a message must be in agreement about who will adjudicate abuse reports, so some process built on top of end-to-end encrypted protocol messages or out-of-band communication should be used to determine this. Since our syntax has the choice of moderator made on the sender device (it is an input to Send), a receiving client who gets a message not assigned to a moderator accepted by the device's user can drop the message before it is shown to the human user.

5. Basic Scheme

Our basic scheme is closely modeled after plain franking, and achieves robust performance at the cost of moderator privacy. The platform holds no key material of its own, and each moderator generates a MAC key $sk_{m,p}$ to share with the platform, as well as a public-key encryption key pair (pk_m, sk_m) . The process of sending a message is identical to plain franking; the sender still produces a ciphertext c_1 and a commitment c_2 to the plaintext message sent. We only require that the sender additionally informs the platform of the identity i of the target moderator. The platform looks up the keys $sk_{m,p}$ and pk_m corresponding to moderator i , and uses these to compute a MAC σ_{pt} on c_2 and the message context ctx . The platform then computes σ as the encryption of σ_{pt} under the moderator's public key pk_m , and sends (σ, ctx, i) to the receiver along with the sender's (c_1, c_2) .

To read a message, the receiver decrypts c_1 to get the message plaintext and commitment randomness r used to generate c_2 . After validating the commitment, and confirming that the choice of moderator is acceptable, the receiver

displays the message to the end user, and stores the message m , choice of moderator i , commitment c_2 , commitment randomness r , message context ctx , and encrypted moderator MAC σ .

In the case that a message is reported, this data is collected into a report for the target moderator. The moderator verifies the commitment, and decrypts and verifies the MAC tag. If these tests pass, the moderator accepts the report and takes action on the reported message and message context, in line with its moderation policies.

Construction 5.1 (Basic scheme). Our "Basic" third-party franking scheme with security parameter λ appears in Figure 2 and makes use of the following primitives:

- An existentially unforgeable MAC scheme $MAC = (KeyGen, Sign, Verify)$.
- A CPA-secure symmetric encryption scheme (Enc, Dec) .
- A binding and hiding commitment scheme $Com = (Commit, Open)$.
- A CPA-secure public key encryption scheme $PKE = (KeyGen, Enc, Dec)$.

In Appendix C, we prove the following theorem by separately proving that each desired security property holds.

Theorem 5.2 (Informal). *The basic scheme is a third-party franking scheme that satisfies accountability, unforgeability, deniability, subpoena safety, and moderator binding.*

6. Achieving Moderator Privacy

We now turn our attention to adding support for moderator privacy to third-party franking. This section introduces two new schemes: Mod-Priv-1 and Mod-Priv-2. The first offers lower computation and communication costs for small numbers of registered moderators, but its costs increase linearly with the number of moderators. The second has higher costs at baseline, but its performance is independent of the number of registered moderators. This section describes both schemes, and the concrete tradeoffs between them are explored in Section 7.

6.1. Moderator Privacy with Linear Costs

The high-level approach of the Mod-Priv-1 scheme is to build on the basic scheme and have the platform preemptively compute MAC tags σ_i for all moderators for every message, since it does not know which moderator is the intended one. Then, by encrypting all the tags under the key of the intended moderator, it renders tags for other moderators useless. The challenge, then, is to find a way for the platform to blindly encrypt the tags σ_i to the intended moderator, producing the desired ciphertext without knowing which moderator's public key it used.

To make this intuition work, we introduce two cryptographic ideas. The first is a modification of ideas from proxy re-encryption to allow the platform to blindly encrypt the MAC tags to the intended moderator. The second takes

$\text{SetupPlatform}(1^\lambda)$ $sk_p, k_{reg} \leftarrow (\varepsilon, \varepsilon)$ return sk_p, k_{reg}	$\text{SetupMod}(k_{reg}, 1^\lambda)$ $sk_{m,p} \leftarrow \text{MAC.KeyGen}(1^\lambda)$ $pk_m, sk_m \leftarrow \text{PKE.KeyGen}(1^\lambda)$ return $sk_{m,p}, pk_m, sk_m$	$\text{Send}(k, m, i, pk_{m,i})$ $r \xleftarrow{\mathcal{R}} \mathcal{R}$ $c_1 \leftarrow \text{Enc}(k, (m, r))$ $c_2 \leftarrow \text{Com.Commit}(m, r)$ $ad \leftarrow i$ return c_1, c_2, ad
$\text{Process}(k_p, ks, c_1, c_2, ad, ctx)$ $i \leftarrow ad$ $sk_{m,p,i}, pk_{m,i} \leftarrow ks[i]$ $\sigma_{pt} \leftarrow \text{MAC.Sign}(sk_{m,p,i}, (c_2, ctx))$ $\sigma \leftarrow \text{PKE.Enc}(pk_{m,i}, \sigma_{pt})$ $st \leftarrow (ctx, i)$ return σ, st	$\text{Read}(k, pks, c_1, c_2, \sigma, st)$ $ctx, i \leftarrow st$ $m, r \leftarrow \text{Dec}(k, c_1)$ assert $\text{Com.Open}(c_2, m, r)$ $rd \leftarrow (r, c_2, ctx, \sigma)$ return m, i, rd $\text{ReportGen}(m, rd)$ $report \leftarrow rd$ return $report$	$\text{Moderate}(sk_m, sk_{m,p}, m, report)$ $r, c_2, ctx, \sigma \leftarrow report$ $\sigma_{pt} \leftarrow \text{PKE.Dec}(sk_m, \sigma)$ if $\text{Com.Open}(c_2, m, r)$: if $\text{MAC.Verify}(sk_{m,p}, (c_2, ctx), \sigma_{pt})$: return ctx return $\perp$

Figure 2: Basic Scheme (Construction 5.1).

$\text{KeyGen}(1^\lambda)$ $x \xleftarrow{\mathcal{R}} \mathbb{Z}_q$ $sk, pk \leftarrow (x, g^x)$ return sk, pk	$\text{ReKeyGen}(sk_1, sk_2)$ $k_{1 \rightarrow 2} \leftarrow sk_2 / sk_1$ return $k_{1 \rightarrow 2}$
$\text{MaskKey}(pk_1)$ $s \xleftarrow{\mathcal{R}} \mathbb{Z}_q$ $epk \leftarrow pk_1^s$ return s, epk	$\text{Enc}(pk, m)$ $r \xleftarrow{\mathcal{R}} \mathbb{Z}_q$ $c_1, c_2 \leftarrow (g^r m, pk^r)$ return c_1, c_2
$\text{ReEnc}(k_{1 \rightarrow 2}, c)$ $c_1, c_2 \leftarrow c$ $c'_2 \leftarrow c_2^{k_{1 \rightarrow 2}}$ return c_1, c'_2	$\text{Dec}(sk, c)$ $c_1, c_2 \leftarrow c$ $m \leftarrow c_1 / (c_2^{1/sk})$ return m
$\text{Verify}(epk, k_{e \rightarrow 2}, pk_2)$ return $(epk^{k_{e \rightarrow 2}} = pk_2)$	

Figure 3: Proxy re-encryption subroutines used by our Mod-Priv-1 and Mod-Priv-2 schemes (Constructions 6.1 and 6.3).

advantage of the structure of the MACs used (HMAC in practice) to only need a single block of public key encryption rather than using hybrid encryption to encrypt a vector of all moderator MACs.

Re-encryption for blind processing. In our scheme, moderator setup has each moderator generate two different encryp-

tion key pairs pk_1, sk_1, pk_2, sk_2 as well as a (bidirectional) proxy re-encryption key $k_{1 \rightarrow 2}$ to go between them. The moderator then discards sk_1 , which is no longer needed.

When sending a message, clients mask the public key pk_1 for the intended moderator with a randomly chosen masking value s , and they also mask the key $k_{1 \rightarrow 2}$ with s . This has the effect of producing an encryption key epk and re-encryption key $k_{e \rightarrow 2}$, where epk appears uniformly random, but $k_{e \rightarrow 2}$ allows for re-encryption to pk_2 for the chosen moderator. The platform is sent epk for encrypting the values of σ_i in ciphertext c_3 , and the re-encryption key $k_{e \rightarrow 2}$ is included inside the ciphertext c_1 sent to the message recipient.

A message recipient can check that $k_{e \rightarrow 2}$ actually does convert the public key epk to the intended pk_2 , and then it can re-encrypt c_3 to pk_2 if the user wishes to make a report. The proxy re-encryption operations we use are formalized in Figure 3.

Reducing public key costs. As described thus far, in a setting with N moderators, the platform needs to encrypt N MACs $\sigma_1, \dots, \sigma_N$ using the proxy re-encryption scheme. This can be done efficiently using a standard hybrid encryption approach where the platform encrypts a random symmetric key, which is in turn used to encrypt all the σ_i values. However, this sets up an unfortunate tradeoff. A single encryption of all the MAC tags would make the size of reports to moderators quite large, due to the number of unnecessary tags communicated in the ciphertext. On the other hand, encrypting each tag in a separate ciphertext so that a client can drop the unnecessary ones before reporting means that each tag requires a separate and more expensive

SetupPlatform (1^λ) $sk_p, k_{reg} \leftarrow (\varepsilon, \varepsilon)$ return sk_p, k_{reg}	SetupMod ($k_{reg}, 1^\lambda$) $sk_{m,p} \leftarrow \text{MAC.KeyGen}(1^\lambda)$ $pk_1, sk_1 \leftarrow \text{RE.KeyGen}(1^\lambda)$ $pk_2, sk_2 \leftarrow \text{RE.KeyGen}(1^\lambda)$ $k_{1 \rightarrow 2} \leftarrow \text{RE.ReGen}(sk_1, sk_2)$ $sk_m \leftarrow sk_2$ $pk_m \leftarrow (pk_1, pk_2, k_{1 \rightarrow 2})$ return $sk_{m,p}, sk_m, pk_m$	Send ($k, m, i, pk_{m,i}$) $pk_1, \cdot, k_{1 \rightarrow 2} \leftarrow pk_{m,i}$ $s, epk \leftarrow \text{RE.MaskKey}(pk_1)$ $k_{e \rightarrow 2} \leftarrow \text{RE.ReKeyGen}(s, k_{1 \rightarrow 2})$ $r \xleftarrow{\mathcal{R}}$ $c_1 \leftarrow \text{Enc}(k, (m, i, r, k_{e \rightarrow 2}))$ $c_2 \leftarrow \text{Com.Commit}(m, r)$ $ad \leftarrow epk$ return c_1, c_2, ad
Process ($sk_p, ks, c_1, c_2, ad, ctx$) $\vec{\sigma} \leftarrow []$ $r' \xleftarrow{\mathbb{R}} \mathbb{G}$ for $i \in \{1, \dots, N\}$: $sk_{m,p} \leftarrow ks[i]$ $\sigma_i \leftarrow \text{MAC.Sign}(sk_{m,p}, (c_2, r', ctx))$ $\vec{\sigma}[i] \leftarrow \sigma_i$ $epk \leftarrow ad$ $c_3 \leftarrow \text{RE.Enc}(epk, r')$ $st \leftarrow (c_3, epk, ctx)$ return $\vec{\sigma}, st$	Read ($k, pks, c_1, c_2, \vec{\sigma}, st$) $c_3, epk, ctx \leftarrow st$ $m, i, r, k_{e \rightarrow 2} \leftarrow \text{Dec}(k, c_1)$ assert $\text{Com.Open}(c_2, m, r)$ $\cdot, pk_2, \cdot \leftarrow pks[i]$ assert $\text{RE.Verify}(epk, k_{e \rightarrow 2}, pk_2)$ $\sigma \leftarrow \vec{\sigma}[i]$ $rd \leftarrow (r, c_2, ctx, \sigma, k_{e \rightarrow 2}, c_3)$ return m, i, rd ReportGen (m, rd) $r, c_2, ctx, \sigma, k_{e \rightarrow 2}, c_3 \leftarrow rd$ $c'_3 \leftarrow \text{RE.ReEnc}(k_{e \rightarrow 2}, c_3)$ $report \leftarrow (r, c_2, c'_3, ctx, \sigma)$ return $report$	Moderate ($sk_m, sk_{m,p}, m, report$) $r, c_2, c'_3, ctx, \sigma \leftarrow report$ $r' \leftarrow \text{RE.Dec}(sk_m, c'_3)$ if $\text{Com.Open}(c_2, m, r)$: if $\text{MAC.Verify}(sk_{m,p}, (c_2, r', ctx), \sigma)$: return ctx return $\perp$

Figure 4: Scheme Mod-Priv-1 providing moderator privacy (Construction 6.1).

public key encryption.

We show how we can get the best of both worlds by taking advantage of the randomness properties of a MAC like HMAC. When computing MACs, the platform samples a new random value r' , which it includes as part of the message being tagged when computing each tag σ_i . Then, instead of encrypting the σ_i tags themselves, the platform only encrypts r' using a single public-key encryption, and it sends the tags σ_i in the clear to the message recipient.

The standard MAC security definition cannot be used to prove this scheme secure, because it is possible that an unintended moderator j who receives the tag σ_j could verify part of the message, even if it does not know the input r' . However, if the MAC behaves like a random function, i.e., we model it as a random oracle, it is impossible to recompute and verify a MAC tag without knowing the full input. Thus, hiding only r' suffices to render the platform-computed tags useless without needing to actually encrypt any of them.

We now formally state our construction of Mod-Priv-1.

Construction 6.1 (Mod-Priv-1). Our third-party franking

scheme *Mod-Priv-1* with security parameter λ appears in Figure 4 with subroutines shown in Figure 3 and makes use of the following primitives.

- A group $\mathbb{G}$ of prime order q with generator g where the DDH problem is hard.
- An existentially unforgeable MAC scheme $\text{MAC} = (\text{KeyGen}, \text{Sign}, \text{Verify})$. We also model the Sign algorithm as a random oracle.
- A CPA-secure symmetric encryption scheme (Enc, Dec) .
- A binding and hiding commitment scheme $\text{Com} = (\text{Commit}, \text{Open})$.

In Appendix D, we prove the following theorem by separately proving that each desired security property holds.

Theorem 6.2 (Informal). *The scheme Mod-Priv-1 is a third-party franking scheme that satisfies accountability, unforgeability, deniability, subpoena safety, moderator privacy, and moderator binding.*

SetupPlatform (1^λ) $sk_p \xleftarrow{R} \mathbb{Z}_p$ $k_{reg} \leftarrow g_2^{1/sk_p}$ return sk_p, k_{reg}	SetupMod ($k_{reg}, 1^\lambda$) $k_m \xleftarrow{R} \mathbb{Z}_p$ $pk_1, sk_1 \leftarrow \text{RE.KeyGen}(1^\lambda)$ $pk_2, sk_2 \leftarrow \text{RE.KeyGen}(1^\lambda)$ $k_{1 \rightarrow 2} \leftarrow \text{RE.ReGen}(sk_1, sk_2)$ $pk_{proc} \leftarrow k_{reg}^{k_m}$ $sk_m \leftarrow (sk_2, k_m)$ $pk_m \leftarrow (pk_1, pk_2, k_{1 \rightarrow 2}, pk_{proc})$ return $(\varepsilon, sk_m, pk_m)$	Send ($k, m, i, pk_{m,i}$) $pk_1, \cdot, k_{1 \rightarrow 2}, \cdot \leftarrow pk_{m,i}$ $s, epk \leftarrow \text{RE.MaskKey}(pk_1)$ $k_{e \rightarrow 2} \leftarrow \text{RE.ReKeyGen}(s, k_{1 \rightarrow 2})$ $r \xleftarrow{R} \mathcal{R}$ $c_1 \leftarrow \text{Enc}(k, (m, i, r, k_{e \rightarrow 2}))$ $c_2 \leftarrow \text{Com.Commit}(m, r)$ $ad \leftarrow epk$ return c_1, c_2, ad
Process ($sk_p, ks, c_1, c_2, ad, ctx$) $r' \xleftarrow{R} \mathbb{G}$ $epk \leftarrow ad$ $\sigma \leftarrow H(c_2, r', ctx)^{sk_p}$ $c_3 \leftarrow \text{RE.Enc}(epk, r')$ $st \leftarrow (c_3, epk, ctx)$ return σ, st	Read ($k, pks, c_1, c_2, \sigma, st$) $m, i, r, k_{e \rightarrow 2} \leftarrow \text{Dec}(k, c_1)$ assert $\text{Com.Open}(m, r, c_2)$ $c_3, epk, ctx \leftarrow st$ $\cdot, pk_2, \cdot, pk_{proc} \leftarrow pks[i]$ assert $\text{RE.Verify}(epk, k_{e \rightarrow 2}, pk_2)$ $rd \leftarrow (r, c_2, ctx, \sigma, pk_{proc}, k_{e \rightarrow 2}, c_3)$ return m, i, rd ReportGen (m, rd) $r, c_2, ctx, \sigma, pk_{proc}, k_{e \rightarrow 2}, c_3 \leftarrow rd$ $\sigma' \leftarrow e(\sigma, pk_{proc})$ $c'_3 \leftarrow \text{RE.ReEnc}(k_{e \rightarrow 2}, c_3)$ $report \leftarrow (r, c_2, ctx, \sigma', c'_3)$ return $report$	Moderate ($sk_m, sk_{m,p}, m, report$) $r, c_2, ctx, \sigma', c'_3 \leftarrow report$ $sk_2, k_m \leftarrow sk_m$ $r' \leftarrow \text{RE.Dec}(sk_2, c'_3)$ if $\text{Com.Open}(m, r, c_2)$: if $\sigma' = e(H(c_2, r', ctx)^{k_m}, g_2)$: return ctx return $\perp$

Figure 5: Scheme Mod-Priv-2 providing moderator privacy with costs independent of the number of moderators (Construction 6.3).

6.2. Moderator Privacy with Constant Costs

Mod-Priv-2 iterates on the design of Mod-Priv-1 and shows how to achieve computation and communication costs independent of the number of registered moderators. Whereas Mod-Priv-1 produces and then “encrypts” MAC tags σ_i for each moderator, using proxy re-encryption tools to send the ciphertext to the correct moderator, Mod-Priv-2 goes further and creates a single MAC which gets converted to a MAC for the intended moderator by using different proxy re-encryption tools. This results in a third-party franking scheme that provides moderator privacy while incurring costs completely independent of the number of registered moderators.

Proxy re-MACing. The main new idea in Mod-Priv-2 is to replace the generic MAC used in Mod-Priv-1 with the specific MAC function

$$\text{MAC.Sign}(k, x) = H(x)^k,$$

allowing us to take advantage of the additional algebraic structure of this MAC scheme. Specifically, we have the

platform compute this MAC σ using a key sk_p known only to the platform in an elliptic curve group $\mathbb{G}_1$ that supports an efficient pairing operation with another group $\mathbb{G}_2$. We also have moderators pick their own secret keys k_m . When a new moderator is registered with the platform, the platform sends it the registration key $k_{reg} = g_2^{1/sk_p}$, allowing the moderator to compute a client-side processing key

$$pk_{proc} = k_{reg}^{k_m} = g_2^{k_m/sk_p}.$$

By computing the pairing $\sigma' \leftarrow e(\sigma, pk_{proc})$, the message recipient effectively converts from the MAC σ under the platform key sk_p in $\mathbb{G}_1$ into another MAC σ' under the moderator key k_m in the pairing target group $\mathbb{G}_2$. We prove the security of the resulting “proxy re-MAC” scheme under a variant of the computational bilinear Diffie Hellman (BDH) assumption which is implied by the original assumption. See Appendix B for details.

Much like Mod-Priv-1, we also use proxy re-encryption to encrypt randomness r' which is incorporated as an input to H . This ensures the subpoena safety of the scheme, because the message recipient’s value of σ cannot be verified

by using only sk_p without access to r' , which is encrypted to the target moderator.

The formal description of Mod-Priv-2 appears below.

Construction 6.3 (Mod-Priv-2). Our third-party franking scheme *Mod-Priv-2* with security parameter λ appears in Figure 5 with subroutines shown in Figure 3 and makes use of the following primitives.

- A group $\mathbb{G}$ of prime order q with generator g where the DDH problem is hard.
- Groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order p with generators g_1, g_2, g_T , respectively, and a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ where the computational bilinear Diffie Hellman (BDH) problem is hard.
- An existentially unforgeable MAC scheme $\text{MAC} = (\text{KeyGen}, \text{Sign}, \text{Verify})$. We also model the Sign algorithm as a random oracle.
- A CPA-secure symmetric encryption scheme (Enc, Dec) .
- A binding and hiding commitment scheme $\text{Com} = (\text{Commit}, \text{Open})$.
- A hash function $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$, modeled as a random oracle.

In Appendix E, we prove the following theorem by separately proving that each desired security property holds.

Theorem 6.4 (Informal). *The scheme Mod-Priv-2 is a third-party franking scheme that satisfies accountability, unforgeability, deniability, subpoena safety, moderator privacy, and moderator binding.*

7. Implementation and Evaluation

This section reports on the implementation and evaluation of our three verifiable abuse reporting schemes.

7.1. Implementation

We implemented all three of our abuse reporting schemes. Our implementation of El-Gamal encryption is built on the Ristretto group abstraction offered by the `curve25519_dalek` library [13]. We use the `blstrs` library [1] for access to BLS-381 curves and pairing operations, and all other cryptographic primitives come from `openssl`. We instantiate our MAC and commitment schemes with HMAC-SHA256 and our symmetric encryption scheme with AES256-GCM. We instantiate the hash function H with SHA256 throughout, except when mapping bytes to an element of $\mathbb{G}$ (the Ristretto group over `curve25519`) for El-Gamal encryption in the basic scheme, where we use SHA512. These hash functions are known to be indistinguishable from a random oracle for constant-sized inputs [12], [28], which is how they are used here.

Our El-Gamal encryption implementation interprets its inputs as elements of $\mathbb{G}$ directly, so the basic scheme needs to map $\sigma_{pt} \in \{0, 1\}^*$ to an element of $\mathbb{G}$. We thus map the MAC output σ_{pt} from bits to an element of $\mathbb{G}$ for encryption

purposes. Since we can't easily convert back to bits in the library we use, we convert the MAC output to a group element for MAC verification as well.

7.2. Performance Evaluation

We perform our evaluation on an 11th gen Intel i7 processor 3.6 Ghz, where we evaluate our schemes using a message size of 100 bytes. We re-ran the evaluations of prior works on the same machine using the benchmarks provided by the authors. We also compared to an implementation of Messenger's message franking algorithm (plain franking) which we re-implemented ourselves.

Computation costs. Figure 6 show the computation time for each operation in each of our schemes. The Basic scheme has the best performance because the client-side operations are identical to those of plain franking. Message processing and moderation incur the added cost of a single public-key encryption and decryption, respectively.

In order to provide moderator privacy, Mod-Priv-1 and Mod-Priv-2 incur somewhat higher costs on both the client and the server. This cost arises from the larger quantity of group operations required to handle the proxy re-encryption and proxy re-MAC operations of those schemes. The two schemes have similar client-side costs to send and read messages, but Mod-Priv-2 incurs significantly higher costs to make and verify reports due to computing pairings. Message processing on the server is initially faster in Mod-Priv-1, but becomes slower as the number of moderators increases, eventually becoming more expensive than Mod-Priv-2. Figure 7 shows platform processing time for the three schemes.

Communication costs. Figure 8 shows the communication overhead of our schemes beyond the cost of sending a message using standard E2EE messaging. Most of our communication overhead stems from the various group elements communicated. Our schemes use 32-byte commitments, 32-byte MAC digests, 32-byte commitment randomness, and 4-byte moderator IDs. El-Gamal ciphertexts consist of 2 Ristretto Points of 32 bytes each. Scalars are also 32 bytes in our implementation. For BLS curves, the group elements take up 48 bytes, 96 bytes, and 288 bytes for groups $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ respectively. Finally, in the Mod-Priv-1, the number of tags produced grows linearly with the number of registered moderators, requiring more communication than Mod-Priv-2 even when there are only 2 moderators.

Comparison to prior work. We compare the computation and communication costs of our schemes with prior work in the message franking literature. In particular, we compare to Asymmetric Message Franking (AMF) [36], Hecate [21], and plain franking. We use the `amaze` implementation of AMF, which is much faster than the Python implementation in the original paper [30]. It is important to note that our work and AMF/Hecate target fairly different use cases: those schemes target abuse reporting for a single third-party moderator for metadata-hiding messaging. Our

	Preprocessing (μs)	Send (μs)	Process (μs)	Read (μs)	Report (μs)	Moderate (μs)
Basic	-	1.25	40.89	0.78	-	35.83
Mod-Priv-1 {2}	-	33.45	47.79	24.36	28.42	36.49
Mod-Priv-1 {64}	-	33.59	70.97	23.88	33.09	37.70
Mod-Priv-2	-	34.14	166.18	23.69	764.15	719.49
AMF	-	231.74	-	225.05	-	224.89
Hecate	29.32	16.63	15.70	97.87	-	103.48
Plain Franking	-	0.99	0.41	0.89	-	0.72

Figure 6: Computation time for 100 byte messages. For Mod-Priv-1, $\{N\}$ represents the number of moderators registered. Only Hecate requires per-message preprocessing, and the time shown represents preprocessing time per message.

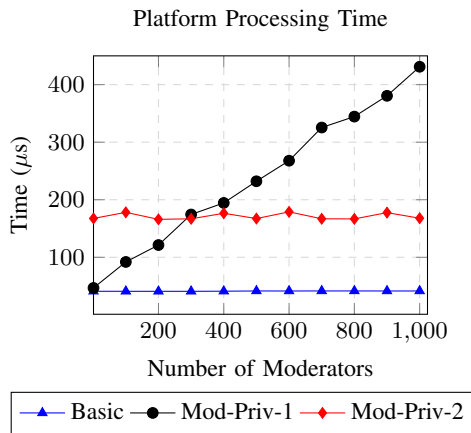


Figure 7: Server processing time of our schemes with respect to the number of registered moderators.

	Sending	Receiving	Reporting
Basic	68 B	132 B	128 B
Mod-Priv-1	132 B	$32N + 196$ B	160 B
Mod-Priv-2	132 B	244 B	416 B
AMF	489 B	489 B	489 B
Hecate	380 B	484 B	380 B
Plain Franking	64 B	96 B	96 B

Figure 8: Communication overhead for our schemes and prior work, in bytes. Here, $32N$ represents the 32-byte MAC tag computed for each of the N moderators. Unlike prior works, our constructions explicitly distinguish between data received by a client and data that needs to be stored to make a report. This storage cost is 132 bytes for the Basic scheme, 196 bytes for Mod-Priv-1, and 212 bytes for Mod-Priv-2.

work targets abuse reporting for the case of multiple third-party moderators, but does not consider the metadata-hiding case. Thus the comparison is not quite apples-to-apples, but it provides a sense of how our work compares to other

works offering similar functionalities. Plain franking, which provides the least functionality, provides a comparison for the best-possible performance one could hope for from a franking scheme.

Figure 6 displays the computation costs for each of our algorithms and for each prior work. Our basic scheme is comparable to plain franking for client-side costs to send and receive messages, but it incurs higher costs to process messages on the platform and verify reports on the moderator because of the addition of an El-Gamal encryption/decryption step. The basic scheme has slightly lower overall computation costs on the platform compared to Hecate. Since part of the El-Gamal encryption can be generated in advance, these two schemes have very similar characteristics on the platform side. On the other hand, message sending is $13\times$ faster, receiving is $125\times$ faster, and moderation is $3\times$ faster. The savings compared to AMF are even larger.

Performance of the moderator privacy schemes is more comparable to prior work, with costs across all schemes being on the same order of magnitude and no single scheme dominating the computation time of the others across all protocol steps. Mod-Priv-2 is slower than the rest in the reporting and report verification process due to computing pairings, but fortunately this step of the process only needs to be called in the case a user makes a report, not for all messages sent or received. A high-level intuition for this may be that while plain franking and the basic scheme are both primarily focused on protecting information about message contents, Mod-Priv-1, Mod-Priv-2, AMF, and Hecate also need to protect some kind of non-message information: either the identity of the moderators or the identities of the conversation partners. Hiding this kind of information generally seems to require more expensive tools.

In terms of communication, our basic scheme closely follows the costs of plain franking. The only additions are 4 bytes for identifying which moderator is assigned to a message, and 32 bytes additional overhead for encrypting the MAC tag. This constitutes $6-7\times$ savings compared to Hecate and AMF when sending messages and $3-4\times$ savings for receiving and reporting messages. Mod-Priv-1 and Mod-

Priv-2 also offer considerable savings over Hecate and AMF for most operations. The exception is the reporting cost of Mod-Priv-2, which requires sending an element of $\mathbb{G}_T$. This cost is slightly higher than the reporting cost in Hecate but still lower than AMF. The Mod-Priv-1 scheme has costs for sending a message from the platform to a message recipient that increase with the number of moderators, and the crossover point where it has the same communication cost as Hecate is the case of 9 moderators. A comparison of communication costs appears in Figure 8.

8. Conclusion

We have presented a new approach to abuse reporting for E2EE messaging, third-party franking, that allow users to choose which of a list of platform-vetted moderators are responsible for handling reports of their messages. We have explored the security goals relevant to this new setting and proposed three third-party franking schemes that provide a range of performance and security tradeoffs. Our schemes offer performance that matches and sometimes exceeds that of prior works that only support a single moderator.

The introduction of multiple moderators into the abuse reporting process also offers a number of possibilities for future work. For example, the question of how to offer third-party franking for metadata-hiding messaging remains open. We also believe that our contributions can be extended to support both stronger security notions and richer functionalities. Two interesting directions would be to consider accountability definitions where the platform aids message senders in avoiding abuse reporting mechanisms [17] and to explore the setting where each message is assigned multiple potential moderators, any one of which could be asked to verify a report of that message. We hope that continued exploration of new technologies for abuse reporting will equip platforms with a more flexible toolkit and more varied policy options for protecting their users.

Ethical Considerations

The work presented in this paper impacted no existing messaging platforms or real-world users. We believe that this work broadens the space of potential policy options that are available to messaging platforms to enable abuse reporting. Specifically, by allowing for third-party moderators of users' choosing, we give users control and autonomy over what kinds of content they find permissible, and improve the quality of the messaging platform for all users.

References

- [1] blstrs. <https://github.com/filecoin-project/blstrs>, 2020.
- [2] About community notes on x. <https://help.x.com/en/using-x/community-notes>, 2025.
- [3] Introducing community notes. <https://www.meta.com/technologies/community-notes/>, 2025.
- [4] Giuseppe Ateniese, Kevin Fu, Matthew Green, and Susan Hohenberger. Improved proxy re-encryption schemes with applications to secure distributed storage. In *NDSS*, 2005.
- [5] Matt Blaze, Gerrit Bleumer, and Martin Strauss. Divertible protocols and atomic proxy cryptography. In Kaisa Nyberg, editor, *EUROCRYPT*, 1998.
- [6] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, 2001.
- [7] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. *SIAM J. Comput.*, 32(3):586–615, 2003.
- [8] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *CRYPTO*, 2013.
- [9] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography (version 0.6)*. 2023. <https://cryptobook.us>.
- [10] Melissa Chase, Apoorva Deshpande, Esha Ghosh, and Harjasleen Malvai. Seamless: Secure end-to-end encrypted messaging with less trust. In *ACM CCS*, 2019.
- [11] Brian Chen, Yevgeniy Dodis, Esha Ghosh, Eli Goldin, Balachandhar Kesavan, Antonio Marcedone, and Merry Ember Mou. Rotatable zero knowledge sets - post compromise secure auditable dictionaries with application to key transparency. In *ASIACRYPT*, 2022.
- [12] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-damgård revisited: How to construct a hash function. In *CRYPTO*, 2005.
- [13] Henry de Valence and Isis Agora Lovecruft. curve25519-dalek (version 3.2.1), 2023.
- [14] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryption. In *CRYPTO*, 2018.
- [15] Saba Eskandarian. Abuse reporting for metadata-hiding communication based on secret sharing. *USENIX Security*, 2024.
- [16] Taher El Gamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In *CRYPTO*, 1984.
- [17] Matthew Gregoire, Margaret Pierce, and Saba Eskandarian. Onion franking: Abuse reports for mix-based private messaging. In *NDSS*, 2025.
- [18] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *CRYPTO*, 2017.
- [19] Susan Hohenberger, Guy N. Rothblum, Abhi Shelat, and Vinod Vaikuntanathan. Securely obfuscating re-encryption. *J. Cryptol.*, 24(4):694–719, 2011.
- [20] Yuncong Hu, Kian Hooshmand, Harika Kalidhindi, Seung Jin Yang, and Raluca Ada Popa. Merkle²: A low-latency transparency log system. In *IEEE S&P*, pages 285–303. IEEE, 2021.
- [21] Rawane Issa, Nicolas Alhaddad, and Mayank Varia. Hecate: Abuse reporting in secure messengers with sealed sender. *USENIX Security*, 2022.
- [22] Anca-Andreea Ivan and Yevgeniy Dodis. Proxy cryptography revisited. In *NDSS*, 2003.
- [23] Antoine Joux. A one round protocol for tripartite diffie-hellman. In *ANTS*, 2000.
- [24] Antoine Joux. A one round protocol for tripartite diffie-hellman. *J. Cryptol.*, 17(4):263–276, 2004.
- [25] Julia Len, Melissa Chase, Esha Ghosh, Daniel Jost, Balachandhar Kesavan, and Antonio Marcedone. ELEKTRA: efficient lightweight multi-device key transparency. In *ACM CCS*, 2023.
- [26] Julia Len, Melissa Chase, Esha Ghosh, Kim Laine, and Radames Cruz Moreno. OPTIKS: an optimized key transparency system. In *USENIX Security*, 2024.

- [27] Harjasleen Malvai, Lefteris Kokoris-Kogias, Alberto Sonnino, Esha Ghosh, Ercan Oztürk, Kevin Lewi, and Sean F. Lawlor. Parakeet: Practical key transparency for end-to-end encrypted messaging. In *NDSS*, 2023.
- [28] Ueli M. Maurer, Renato Renner, and Clemens Holenstein. Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In *TCC*, 2004.
- [29] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: bringing key transparency to end users. In *USENIX Security*, 2015.
- [30] Sanketh Menda and Michael Rosenberg. amaze. <https://github.com/sgmenda/amaze>, 2022.
- [31] Meta. Messenger end-to-end encryption overview, December 2023.
- [32] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and KDCs. In *EUROCRYPT*, 1999.
- [33] Trevor Perrin and Moxie Marlinspike. The double ratchet algorithm. <https://signal.org/docs/specifications/doubleratchet/>, 2016.
- [34] Sarah Scheffler and Jonathan R. Mayer. Sok: Content moderation for end-to-end encryption. *Proc. Priv. Enhancing Technol.*, 2023(2):403–429, 2023.
- [35] Nirvan Tyagi, Ben Fisch, Andrew Zitek, Joseph Bonneau, and Stefano Tessaro. Versa: Verifiable registries with efficient client audits from RSA authenticated dictionaries. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS*, 2022.
- [36] Nirvan Tyagi, Paul Grubbs, Julia Len, Ian Miers, and Thomas Ristenpart. Asymmetric message franking: Content moderation for metadata-private end-to-end encryption. In *CRYPTO*, 2019.
- [37] Ioanna Tzialla, Abhiram Kothapalli, Bryan Parno, and Srinath T. V. Setty. Transparency dictionaries with succinct proofs of correct operation. In *NDSS*, 2022.

Appendix A. Background

This section provides additional background on tools used in this paper.

Pairings. A pairing [6], [7], [23], [24] is an efficiently computable mapping $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that maps elements of elliptic curve groups $\mathbb{G}_1, \mathbb{G}_2$ of order p with generators $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ to elements of a target group $\mathbb{G}_T$. The pairing must satisfy *non-degeneracy*, requiring that $e(g_1, g_2)$ is not the identity in $\mathbb{G}_T$, as well as the following *bilinearity* property: for all $a, b \in \mathbb{Z}_p$, it holds that

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}.$$

We will use pairing groups where we assume the computational bilinear Diffie-Hellman problem is hard. See Appendix B for a discussion of this assumption and the variant which we use.

Proxy re-encryption. Proxy re-encryption (PRE) allows a party to give another party the power to redirect ciphertexts intended for a particular recipient to another recipient [4], [5], [19], [22]. Our two schemes that provide moderator hiding will use PRE in different ways, but the underlying ideas are similar. We briefly sketch them here for the sake of completeness, but see Figure 3 for a complete description of our re-encryption scheme.

The high-level idea of the PRE we use is to start with a variant of El-Gamal encryption [16] where the ciphertext is a tuple of elements of a group $\mathbb{G}$ with generator g . For a message m , randomness r , and public key $\text{pk}_1 = g^{\text{sk}_1}$, the ciphertext is $(c_1, c_2) = (g^r m, \text{pk}_1^r)$. Note that this c_1, c_2 is unrelated to the c_1, c_2 used for message franking above. Now, in order to go from a ciphertext encrypted under pk_1 to a ciphertext encrypted under a different pk_2 , the re-encrypting party needs access to a re-encryption key $k_{1 \rightarrow 2} = \text{sk}_2 / \text{sk}_1$. Then re-encryption can be accomplished by computing

$$c'_2 \leftarrow c_2^{k_{1 \rightarrow 2}} = (\text{pk}_1^r)^{k_{1 \rightarrow 2}} = (g^{\text{sk}_1 r})^{k_{1 \rightarrow 2}} = g^{\text{sk}_2 r} = \text{pk}_2^r.$$

This means (c_1, c'_2) is an encryption of m under pk_2 .

As described thus far, re-encryption is a bidirectional process, as $k_{1 \rightarrow 2}^{-1} = k_{2 \rightarrow 1}$ can be used to go from an encryption under pk_2 to an encryption under pk_1 . This process can be made unidirectional using pairings. Given a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, we compute c_1, c_2 as described above using $c_1 \in \mathbb{G}_T$ and $c_2 \in \mathbb{G}_1$, and we set $k_{1 \rightarrow 2} = g_2^{\text{sk}_2 / \text{sk}_1}$. Now we can compute

$$c'_2 \leftarrow e(c_2, k_{1 \rightarrow 2}) = e(g_1^{\text{sk}_1 r}, g_2^{\text{sk}_2 / \text{sk}_1}) = e(g_1, g_2)^{\text{sk}_2 r},$$

which can be used to recover m from c_1 as $m = c_1 / (c'_2)^{\text{sk}_2^{-1}}$.

We do not use PRE techniques in exactly the way described here, but our ideas are inspired by these ideas. In particular, we use the bidirectional PRE approach to mask a moderator’s public encryption key (and therefore the identity of the chosen moderator) and the unidirectional PRE idea to convert a MAC under the platform key to a MAC under the chosen moderator’s key.

Appendix B. Definitions

This section formally describes the security properties required of our underlying cryptographic primitives and presents the formal definitions described informally in Section 3.2.

Standard cryptographic assumptions and definitions. Unless otherwise specified, we use standard security definitions for all cryptographic primitives present in our schemes. This includes CPA-security of public and symmetric-key encryption, the binding and hiding properties of commitment schemes, and the DDH assumption [9]. To make our proofs clearer to follow, we use the variant of the standard existential unforgeability definition for MACs where the adversary is given access to a MAC verification oracle in addition to a signing oracle, which has a standard reduction from existential unforgeability without verification queries at the cost of a multiplicative $1/Q$ factor loss in the adversary’s advantage, where Q is the number of verification queries.

Bilinear Diffie Hellman. The standard computational bilinear Diffie Hellman problem (for random bases) in pairing groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of order p has the challenger sample

$g_1 \xleftarrow{R} \mathbb{G}_1$, $g_2 \xleftarrow{R} \mathbb{G}_2$, $x, y, z \xleftarrow{R} \mathbb{Z}_p$ and send the adversary the values

$$g_1, g_2, X_1 \leftarrow g_1^x, X_2 \leftarrow g_2^x, Y_1 \leftarrow g_1^y, Y_2 \leftarrow g_2^z.$$

The adversary's task is to compute $Z = e(g_1, g_2)^{xyz}$.

We prove the security of our Mod-Priv-1 and Mod-Priv-2 schemes under a variant of this definition where the adversary is given

$$g_1, g_2, X_1 \leftarrow g_1^x, X_2 \leftarrow g_2^{1/x}, Y_1 \leftarrow g_1^y, Y_2 \leftarrow g_2^{z/x},$$

and the adversary's task is to compute $Z = e(g_1, g_2)^{yz}$.

Our modified version of BDH is implied by the standard version by a straightforward reduction which swaps the values of g_2 and g_2^x . The reduction uses an adversary who solves the modified BDH to break the standard BDH by sending the modified BDH adversary its own BDH challenge inputs, except with g_2 and g_2^x swapped. It then passes on the modified BDH adversary's output as its own.

It is easy to see that the reduction's challenge to the modified BDH adversary has the correct distribution. Now, let $\hat{g}_2 = g_2^x$. Since the modified BDH adversary produces a value of $Z = e(g_1, \hat{g}_2)^{yz}$, the bilinearity of the pairing gives

$$\begin{aligned} Z &= e(g_1, \hat{g}_2)^{yz} \\ &= e(g_1, g_2^x)^{yz} \\ &= e(g_1, g_2)^{xyz}, \end{aligned}$$

so Z is in fact the correct solution to the standard BDH problem as well.

Definitions for third-party franking. The remainder of this section formalizes the various security properties required of a third-party franking scheme, as described in Section 3.2.

Definition B.1 (Accountability). We define the third-party franking accountability experiment $\text{ACCT}[\mathcal{A}, \mathcal{F}, N, \lambda]$ in Figure 9 with respect to an efficient adversary $\mathcal{A}$, a third-party franking scheme $\mathcal{F}$, a number of moderators N , and a security parameter λ .

We define the *accountability advantage* of $\mathcal{A}$ as

$$\text{ACCTAdv}(\mathcal{A}, \mathcal{F}, N, \lambda) = \Pr[\text{ACCT}[\mathcal{A}, \mathcal{F}, N, \lambda] = 1].$$

We say that a scheme $\mathcal{F}$ has *accountability* if, for all efficient adversaries $\mathcal{A}$, and all $N \in \mathbb{N}$,

$$\text{ACCTAdv}[\mathcal{A}, \mathcal{F}, N, \lambda] \leq \text{negl}(\lambda).$$

Definition B.2 (Unforgeability). We define the third-party franking unforgeability experiment $\text{FORG}[\mathcal{A}, \mathcal{F}, N, Q_S, Q_F, \lambda]$ in Figure 10 with respect to an efficient adversary $\mathcal{A}$ who makes at most Q_S queries to the $\mathcal{O}_{\text{SendMsg}}$ oracle and Q_F queries to the $\mathcal{O}_{\text{ForgeMsg}}$ oracle, a third-party franking scheme $\mathcal{F}$, a number of moderators N , and a security parameter λ .

We define the *unforgeability advantage* of $\mathcal{A}$ as

$$\begin{aligned} \text{FORGAdv}(\mathcal{A}, \mathcal{F}, N, Q_S, Q_F, \lambda) \\ = \Pr[\text{FORG}[\mathcal{A}, \mathcal{F}, N, Q_S, Q_F, \lambda] = 1]. \end{aligned}$$

$\text{ACCT}[\mathcal{A}, \mathcal{F}, N, \lambda]$

```

 $\text{sk}_p, k_{\text{reg}} \leftarrow \text{SetupPlatform}(1^\lambda)$ 
 $\text{ks}, \text{pks} \leftarrow ([], [])$ 
for  $i \in \{1, \dots, N\}$  :
     $\text{sk}_{m,p,i}, \text{sk}_{m,i}, \text{pk}_{m,i} \leftarrow \text{SetupMod}(k_{\text{reg}}, 1^\lambda)$ 
     $\text{ks.append}((\text{sk}_{m,p,i}, \text{pk}_{m,i}))$ 
     $\text{pks.append}(\text{pk}_{m,i})$ 
 $\text{win} \leftarrow 0$ 
 $\mathcal{A}^{\mathcal{O}_{\text{SendMsg}}(k_{\text{reg}}, \text{pks}, 1^\lambda)}$ 
output win

```

$\mathcal{O}_{\text{SendMsg}}(k, c_1, c_2, \text{ad}, \text{ctx})$

```

 $\sigma, \text{st} \leftarrow \text{Process}(\text{sk}_p, \text{ks}, c_1, c_2, \text{ad}, \text{ctx})$ 
 $m, i, \text{rd} \leftarrow \text{Read}(k, \text{pks}, c_1, c_2, \sigma, \text{st})$ 
 $\text{report} \leftarrow \text{ReportGen}(m, \text{rd})$ 
 $\text{res} \leftarrow \text{Moderate}(\text{sk}_{m,i}, \text{sk}_{m,p,i}, m, \text{report})$ 
if  $(m, i, \text{report}) \neq \perp$  :
    if  $\text{res} \neq \text{ctx} \vee \text{res} = \perp$  :
         $\text{win} \leftarrow 1$ 
return  $m, i, \text{report}, \text{res}$ 

```

Figure 9: Accountability

We say that a scheme $\mathcal{F}$ has *unforgeability* if, for all efficient adversaries $\mathcal{A}$, and all $N, Q_S, Q_F \in \mathbb{N}$,

$$\text{FORGAdv}[\mathcal{A}, \mathcal{F}, N, Q_S, Q_F, \lambda] \leq \text{negl}(\lambda).$$

Definition B.3 (Deniability). We define the third-party franking universal, platform-compromise, and moderator-compromise deniability experiments $\{\text{U}, \text{MC}, \text{PC}\}\text{-DENY}[\mathcal{A}, \mathcal{F}, N, \text{Forge}, b, \lambda]$ in Figure 11 with respect to an efficient adversary $\mathcal{A}$, a third-party franking scheme $\mathcal{F}$, a number of moderators N , a report forgery algorithm Forge , a bit b , and a security parameter λ .

We define the respective *deniability advantages*, $\{\text{U}, \text{MC}, \text{PC}\}\text{-DENYAdv}(\mathcal{A}, \mathcal{F}, N, \text{Forge}, \lambda)$, of $\mathcal{A}$ as

$$\begin{aligned} &|\Pr[\{\text{U}, \text{MC}, \text{PC}\}\text{-DENY}[\mathcal{A}, \mathcal{F}, N, \text{Forge}, 0, \lambda] = 1] - \\ &\Pr[\{\text{U}, \text{MC}, \text{PC}\}\text{-DENY}[\mathcal{A}, \mathcal{F}, N, \text{Forge}, 1, \lambda] = 1]|. \end{aligned}$$

We say that a scheme $\mathcal{F}$ has *deniability* if there exist forgery algorithms Forge for all three deniability games such that, for all efficient adversaries $\mathcal{A}$, and all $N \in \mathbb{N}$,

$$\{\text{U}, \text{MC}, \text{PC}\}\text{-DENYAdv}[\mathcal{A}, \mathcal{F}, N, \text{Forge}, \lambda] \leq \text{negl}(\lambda).$$

Note: In order to reduce redundancy, we have collapsed all three deniability definitions into one figure, and for clarity have color-coded which keys are given to which party. We've also color-coded the differences between the forgery algorithms in our deniability proofs. Throughout this

```

FORG[ $\mathcal{A}, \mathcal{F}, N, Q_S, Q_F, \lambda$ ]


---


 $sk_p, k_{reg} \leftarrow \text{SetupPlatform}(1^\lambda)$ 
 $ks, pks \leftarrow ([], [])$ 
 $target \leftarrow \mathcal{A}()$ 
for  $i \in \{1, \dots, N\}$  :
     $sk_{m,p,i}, sk_{m,i}, pk_{m,i} \leftarrow \text{SetupMod}(k_{reg}, 1^\lambda)$ 
     $ks.append((sk_{m,p,i}, pk_{m,i}))$ 
     $pks.append(pk_{m,i})$ 
    if  $i \neq target$  :
         $\mathcal{A}(sk_{m,p,i}, sk_{m,i})$ 
 $T_{seen} \leftarrow \{\}$ 
 $win \leftarrow 0$ 
 $\mathcal{A}^{\mathcal{O}_{\text{SendMsg}, \text{ForgeMsg}}}(k_{reg}, pks, 1^\lambda)$ 
output  $win$ 



---


 $\mathcal{O}_{\text{SendMsg}}(k, m, ctx)$ 


---


 $c_1, c_2, ad \leftarrow \text{Send}(k, m, target, pk_{m,target})$ 
 $\sigma, st \leftarrow \text{Process}(sk_p, ks, c_1, c_2, ad, ctx)$ 
 $m, i, rd \leftarrow \text{Read}(k, pks, c_1, c_2, \sigma, st)$ 
 $report \leftarrow \text{ReportGen}(m, rd)$ 
 $res \leftarrow \text{Moderate}(sk_{m,i}, sk_{m,p,i}, m, report)$ 
 $T_{seen} \leftarrow T_{seen} \cup \{(m, ctx)\}$ 
return  $c_1, c_2, ad, \sigma, st, report, res$ 



---


 $\mathcal{O}_{\text{ForgeMsg}}(m^*, report^*)$ 


---


 $ctx^* \leftarrow \text{Moderate}(pk_{m,target}, sk_{m,p,target}, m^*, report^*)$ 
if  $ctx^* \neq \perp$  :
    if  $(m^*, ctx^*) \notin T_{seen}$  :
         $win \leftarrow 1$ 
return  $win$ 

```

Figure 10: Unforgeability

paper, universal deniability components are shown in black, moderator-compromise components are shown in blue, and platform-compromise components are shown in yellow. In some cases, the same addition component is present for both moderator- and platform-compromise deniability. In these cases, that component is shown in green.

Definition B.4 (Subpoena Safety). We define the third-party franking subpoena safety experiment $\text{SUBPOENA}[\mathcal{A}, \mathcal{F}, N, Q, b, \lambda]$ in Figure 12 with respect to an efficient adversary $\mathcal{A}$ who makes at most Q calls to the $\mathcal{O}_{\text{SendMsg}}$ oracle, a third-party franking scheme $\mathcal{F}$, a number of moderators N , a bit b , and a security parameter λ . The definition relies on the existence of a Forge algorithm that

```

{U, MC, PC}–DENY[ $\mathcal{A}, \mathcal{F}, N, \text{Forge}, b, \lambda$ ]


---


 $sk_p, k_{reg} \leftarrow \text{SetupPlatform}(1^\lambda)$ 
 $ks, pks, modkeys \leftarrow ([], [], [])$ 
 $target \leftarrow \mathcal{A}(N)$ 
for  $i \in \{1, \dots, N\}$  :
     $sk_{m,p,i}, sk_{m,i}, pk_{m,i} \leftarrow \text{SetupMod}(k_{reg}, 1^\lambda)$ 
     $pks.append(pk_{m,i})$ 
     $ks.append((sk_{m,p,i}, pk_{m,i}))$ 
    if  $i \neq target$  :
         $modkeys.append((sk_{m,p,i}, sk_{m,i}, pk_{m,i}))$ 
    else :
         $modkeys.append((sk_{m,p,i}, \varepsilon, pk_{m,i}))$ 
 $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{SendMsg}}}(sk_p, k_{reg}, modkeys, pk_{m,target}, 1^\lambda)$ 
output  $b'$ 



---


 $\mathcal{O}_{\text{SendMsg}}(k, m, ctx)$ 


---


 $c_1, c_2, ad \leftarrow \text{Send}(k, m, target, pk_{m,target})$ 
 $\sigma, st \leftarrow \text{Process}(sk_p, ks, c_1, c_2, ad, ctx)$ 
 $m', \cdot, rd \leftarrow \text{Read}(k, pks, c_1, c_2, \sigma, st)$ 
 $report_0 \leftarrow \text{ReportGen}(m', rd)$ 
 $report_1 \leftarrow \text{Forge}(pks, target, sk_p, k_{reg}, ks,$ 
     $sk_{m,target}, sk_{target,p}, m, ctx)$ 
return  $report_b$ 

```

Figure 11: Universal deniability is shown in black. Additions necessary for moderator-compromise deniability are shown in blue, those necessary for platform-compromise deniability are shown in yellow, and those necessary for both are shown in green.

satisfies the following requirements.

We define the *subpoena safety advantage* of $\mathcal{A}$, $\text{SUBPOENAAAdv}(\mathcal{A}, \mathcal{F}, N, Q, \lambda)$, as

$$|\Pr[\text{SUBPOENA}[\mathcal{A}, \mathcal{F}, N, Q, 0, \lambda] = 1] - \Pr[\text{SUBPOENA}[\mathcal{A}, \mathcal{F}, N, Q, 1, \lambda] = 1]|.$$

We say that a scheme $\mathcal{F}$ has *subpoena safety* if, for all efficient adversaries $\mathcal{A}$, and all $N, Q \in \mathbb{N}$,

$$\text{SUBPOENAAAdv}[\mathcal{A}, \mathcal{F}, N, Q, \lambda] \leq \text{negl}(\lambda).$$

Definition B.5 (Moderator Privacy). We define the third-party franking moderator privacy experiment $\text{MODPRIV}[\mathcal{A}, \mathcal{F}, N, U, b, \lambda]$ in Figure 13 with respect to an efficient adversary $\mathcal{A}$ who makes at most U calls to the oracle $\mathcal{O}_{\text{NewKey}}$, a third-party franking scheme $\mathcal{F}$, a number of moderators N , a bit b , and a security parameter λ .

We define the *moderator privacy advantage* of $\mathcal{A}$,

```

SUBPOENA[ $\mathcal{A}, \mathcal{F}, N, Q, b, \lambda$ ]
 $sk_p, k_{reg} \leftarrow \text{SetupPlatform}(1^\lambda)$ 
 $ks, pks, \text{modkeys} \leftarrow ([], [], [])$ 
 $\text{target} \leftarrow \mathcal{A}(N)$ 
for  $i \in \{1, \dots, N\}$  :
     $sk_{m,p,i}, sk_{m,i}, pk_{m,i} \leftarrow \text{SetupMod}(k_{reg}, 1^\lambda)$ 
     $ks.append((sk_{m,p,i}, pk_{m,i}))$ 
     $pks.append(pk_{m,i})$ 
    if  $i \neq \text{target}$  :
         $\text{modkeys.append}((sk_{m,p,i}, sk_{m,i}, pk_{m,i}))$ 
    else :
         $\text{modkeys.append}((sk_{m,p,i}, \varepsilon, pk_{m,i}))$ 
 $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{SendMsg}}}(sk_p, k_{reg}, \text{modkeys}, 1^\lambda)$ 
output  $b'$ 



---


 $\mathcal{O}_{\text{SendMsg}}(k, m, \text{ctx})$ 


---


 $c_1, c_2, \text{ad} \leftarrow \text{Send}(k, m, \text{target}, pk_{m,\text{target}})$ 
 $\sigma, \text{st} \leftarrow \text{Process}(sk_p, ks, c_1, c_2, \text{ad}, \text{ctx})$ 
 $\text{received}_0 \leftarrow (c_1, c_2, \sigma, \text{st})$ 
 $\text{received}_1 \leftarrow \text{Forge}(pks, \text{target}, k, m, \text{ctx})$ 
return  $\text{received}_b$ 

```

Figure 12: Subpoena safety.

MODPRIVAdv($\mathcal{A}, \mathcal{F}, N, U, \lambda$), as

$$|\Pr[\text{MODPRIV}[\mathcal{A}, \mathcal{F}, N, U, 0, \lambda] = 1] - \Pr[\text{MODPRIV}[\mathcal{A}, \mathcal{F}, N, U, 1, \lambda] = 1]|.$$

We say that a scheme $\mathcal{F}$ has *moderator privacy* if, for all efficient adversaries $\mathcal{A}$, and all $N, U \in \mathbb{N}$,

$$\text{MODPRIVAdv}[\mathcal{A}, \mathcal{F}, N, U, \lambda] \leq \text{negl}(\lambda).$$

Definition B.6 (Moderator Binding). We define the third-party franking moderator binding experiment $\text{MODBIND}[\mathcal{A}, \mathcal{F}, N, Q, \lambda]$ in Figure 14 with respect to an efficient adversary $\mathcal{A}$ who makes at most Q queries to its $\mathcal{O}_{\text{Mod}}$ oracle, a third-party franking scheme $\mathcal{F}$, a number of moderators N , and a security parameter λ .

We define the *moderator binding advantage* of $\mathcal{A}$, $\text{MODBINDAdv}(\mathcal{A}, \mathcal{F}, N, Q, \lambda)$, as

$$\Pr[\text{MODBIND}[\mathcal{A}, \mathcal{F}, N, Q, \lambda] = 1]$$

We say that a scheme $\mathcal{F}$ has *moderator binding* if, for all efficient adversaries $\mathcal{A}$, and all $N, Q \in \mathbb{N}$,

$$\text{MODBINDAdv}[\mathcal{A}, \mathcal{F}, N, Q, \lambda] \leq \text{negl}(\lambda).$$

Note that our moderator binding definition does not allow for repeated use of the same ctx string. This is because if each message context contains a sender identity and timestamp, then each ctx should be unique. Formally, this is

necessary to prevent attacks where the adversary achieves a trivial win by using the same ctx twice with different values i_1, i_2 and then reusing the report to i_1 for the second report.

Appendix C.

Proof of Security for Basic Scheme

Theorem C.1 (Basic Scheme Accountability). *Assuming that MAC and PKE satisfy perfect correctness, our third-party franking scheme Π_{Basic} , described in Construction 5.1, satisfies accountability (Definition B.1).*

Specifically, for all efficient adversaries $\mathcal{A}$, all $N \in \mathbb{N}$, and any security parameter λ , our scheme Π_{Basic} satisfies

$$\text{ACCTAdv}[\mathcal{A}, \Pi_{\text{Basic}}, N, \lambda] = 0.$$

Proof. For the sake of contradiction, suppose that the adversary reaches the line setting $\text{win} \leftarrow 1$. Then we must have $(m, i, \text{report}) \neq \perp$, and either $\text{res} \neq \text{ctx}$ or $\text{res} = \perp$. Since the experiment honestly executes Read, ReportGen, and Moderate in $\mathcal{O}_{\text{SendMsg}}$, the constraint $(m, i, \text{report}) \neq \perp$ implies that $\text{Com.Open}(c_2, m, r) = 1$. Otherwise, Read would have failed and output $\perp$. Therefore, since commitment opening is a deterministic process, it must also hold that $\text{Com.Open}(c_2, m, r) = 1$ when this is checked in Moderate. This means there are two ways for the constraint $\text{res} \neq \text{ctx} \vee \text{res} = \perp$ to be met:

- 1) $\text{MAC.Verify}(sk_{m,p}, (c_2, \text{ctx}), \sigma_{\text{pt}}) = 0$, or
- 2) $\text{res} \neq \text{ctx}$.

Note that (1) is not possible: since σ_{pt} and σ are computed honestly in Process, this follows by construction of σ_{pt} and correctness of MAC and PKE. Similarly, (2) is not possible since, by the construction of our scheme, the ctx included inside report is the same one used to compute σ inside Process. Therefore it is not possible to reach the line $\text{win} \leftarrow 1$, and our basic scheme has perfect accountability. $\square$

Theorem C.2 (Basic Scheme Unforgeability). *Assuming that MAC satisfies existential unforgeability, and the commitment Com satisfies binding, our third-party franking scheme Π_{Basic} , described in Construction 5.1, satisfies unforgeability (Definition B.2).*

Specifically, for any adversary $\mathcal{A}$ attacking the unforgeability of our scheme Π_{Basic} , for all $N, Q_S, Q_F \in \mathbb{N}$, and for any security parameter λ , there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ attacking the unforgeability and binding of MAC and Com, respectively, such that

$$\begin{aligned} \text{FORGAdv}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q_S, Q_F, \lambda] \\ \leq \text{MACAdv}[\mathcal{B}, \lambda] + \text{BINDAdv}[\mathcal{C}, \lambda]. \end{aligned}$$

Proof. Our proof proceeds as a series of hybrids.

- **H₀**: This is the unforgeability experiment $\text{FORG}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q_S, Q_F, \lambda]$.
- **H₁** This hybrid is identical to the previous one, except we augment the experiment's table T_{seen} to record tuples (m, r, c_2, ctx) , and now check whether $(m^*, \cdot, \cdot, \text{ctx}^*) \in T_{\text{seen}}$ in the ForgeMsg oracle.

MODPRIV[$\mathcal{A}, \mathcal{F}, N, U, b, \lambda$]	$\mathcal{O}_{\text{SendMsg}}(i_k, m, \text{ctx}, i_0, i_1)$	$\mathcal{O}_{\text{Mod}}(k, m, \text{ctx}, i)$
$sk_p, k_{\text{reg}} \leftarrow \mathcal{A}()$ $ks, pks \leftarrow ([], [])$ for $i \in \{1, \dots, N\}$: $sk_{m,p,i}, sk_{m,i}, pk_{m,i} \leftarrow \text{SetupMod}(k_{\text{reg}}, 1^\lambda)$ $ks.append((sk_{m,p,i}, pk_{m,i}))$ $pks.append(pk_{m,i})$ $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{SendMsg}}, \text{NewKey}, \text{Mod}}(ks, 1^\lambda)$ output b'	$c_1, c_2, \text{ad} \leftarrow \text{Send}(\text{keys}[i_k], m, i_b, pk_{i_b})$ return c_1, c_2, ad $\mathcal{O}_{\text{NewKey}}()$ $k \xleftarrow{\mathcal{R}} \mathcal{K}$ $\text{keys.append}(k)$	$c_1, c_2, \text{ad} \leftarrow \text{Send}(k, m, i, pk_{m,i})$ $\sigma, \text{st} \leftarrow \mathcal{A}(c_1, c_2, \text{ad})$ $m', i', \text{rd} \leftarrow \text{Read}(k, pks, c_1, c_2, \sigma, \text{st})$ $\text{report} \leftarrow \text{ReportGen}(m', \text{rd})$ $\text{res} \leftarrow \text{Moderate}(sk_{i'}, sk_{i',p}, m', \text{report})$ return $c_1, c_2, \text{ad}, \sigma, \text{ctx}, \text{st}, \text{report}, \text{res}$

Figure 13: Moderator Privacy. In the $\mathcal{O}_{\text{Mod}}$ oracle, the adversary may supply a symmetric key k used to encrypt the message. We also overload the $\mathcal{O}_{\text{Mod}}$ syntax to allow the adversary to supply an index i_k , which will process the given message with key $\text{keys}[i_k]$ managed by the experiment.

MODBIND[$\mathcal{A}, \mathcal{F}, N, Q, \lambda$]
$sk_p, k_{\text{reg}} \leftarrow \text{SetupPlatform}(1^\lambda)$ $ks, pks \leftarrow ([], [])$ $\text{target} \leftarrow \mathcal{A}(N)$ for $i \in \{1, \dots, N\}$: $sk_{m,p,i}, sk_{m,i}, pk_{m,i} \leftarrow \text{SetupMod}(k_{\text{reg}}, 1^\lambda)$ $ks.append((sk_{m,p,i}, pk_{m,i}))$ $pks.append(pk_{m,i})$ $T_{\text{seen}} \leftarrow \{\}$ $\text{win} \leftarrow 0$ $\mathcal{A}^{\mathcal{O}_{\text{Mod}}}(k_{\text{reg}}, pks, 1^\lambda)$ output win
$\mathcal{O}_{\text{Mod}}(k, m, i, \text{ctx})$
if $\text{ctx} \in T_{\text{seen}}$: return $\perp$ $T_{\text{seen}} \leftarrow T_{\text{seen}} \cup \{\text{ctx}\}$ $c_1, c_2, \text{ad} \leftarrow \text{Send}(k, m, i, pk_{m,i})$ $\sigma, \text{st} \leftarrow \text{Process}(sk_p, ks, c_1, c_2, \text{ad}, \text{ctx})$ $m', \text{report} \leftarrow \mathcal{A}(c_1, c_2, \sigma, \text{st})$ $\text{res} \leftarrow \text{Moderate}(sk_{m,\text{target}}, sk_{m,p,\text{target}}, m', \text{report})$ if $i \neq \text{target} \wedge \text{res} = \text{ctx}$: $\text{win} \leftarrow 1$ return win

Figure 14: Moderator Binding.

This hybrid is indistinguishable from the previous one, as it doesn't result in any change in the adversary's view.

- **H₂** This hybrid is identical to the previous one, except we add an abort condition: the experiment aborts and outputs 0 if, in a call to $\mathcal{O}_{\text{ForgeMsg}}$, $\text{ctx}^* \neq \perp$ (in other words, Moderate succeeded), and $(\cdot, \cdot, c_2^*, \text{ctx}^*) \notin$

T_{seen} , where c_2^* is included in report^* . By the existential unforgeability of the MAC scheme and Lemma C.3, this hybrid is indistinguishable from the preceding one.

- **H₃** This hybrid is identical to the previous one, except we add a new abort condition: the experiment aborts and outputs 0 if, in a call to $\mathcal{O}_{\text{ForgeMsg}}$, $\text{ctx}^* \neq \perp$, and $(m^*, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$. By the binding of the commitment scheme and Lemma C.4, this hybrid is indistinguishable from the preceding one.

We now show that in **H₃**, the adversary has unforgeability advantage 0. For the sake of contradiction, suppose that, at some point in the experiment, $\mathcal{O}_{\text{ForgeMsg}}$ outputs 1. This means that both of the following are true:

- $\text{Moderate}(sk_{m,\text{target}}, sk_{m,p,\text{target}}, m^*, \text{report}^*) \neq \perp$, and
- $(m^*, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$.

But if both of these are satisfied, then this is a contradiction, as the experiment would have aborted and output 0, by the abort condition added in **H₃**. Since $\mathcal{A}$ has no advantage, the proof of the theorem follows from the proofs of the lemmas and the triangle inequality. $\square$

Lemma C.3. Suppose that the advantage of any MAC adversary $\mathcal{B}$ in breaking the existential unforgeability of MAC is at most $\text{MACAdv}[\mathcal{B}, \lambda]$. Then for all efficient adversaries $\mathcal{A}$ who make at most Q_F calls to the $\mathcal{O}_{\text{ForgeMsg}}$ oracle,

$$\left| \Pr[\mathbf{H}_2(\mathcal{A})] - \Pr[\mathbf{H}_1(\mathcal{A})] \right| \leq \text{MACAdv}[\mathcal{B}, \lambda].$$

Proof. We will show that an adversary who can distinguish between the two hybrids can break the unforgeability of the MAC scheme with the same advantage. Suppose that there exists such an adversary $\mathcal{A}$. Then define an adversary $\mathcal{B}$ for the MAC unforgeability experiment as follows. $\mathcal{B}$ simulates the hybrid **H₂** to $\mathcal{A}$ exactly, except for one change: for the target chosen by $\mathcal{A}$, it replaces MAC.Sign and MAC.Verify operations with corresponding queries to its challenger in the MAC unforgeability experiment. At any point, if the abort condition added in **H₂** is met in some $\mathcal{O}_{\text{ForgeMsg}}$ query with report $(r^*, c_2^*, \text{ctx}^*, \sigma^*)$, $\mathcal{B}$ submits $((c_2^*, \text{ctx}^*), \sigma^*)$ as its MAC forgery.

We now argue that $\mathcal{B}$ breaks the existential unforgeability of the MAC with the same probability that $\mathcal{A}$ triggers the abort condition. Since this condition is the only difference between $\mathbf{H}_1$ and $\mathbf{H}_2$, this completes the lemma.

If $\mathcal{A}$ triggers the abort condition in $\mathbf{H}_2$, then it produced values m^* and $\text{report}^* = (r^*, c_2^*, \text{ctx}^*, \sigma^*)$ such that $\text{MAC.Verify}(\text{sk}_{m,p,\text{target}}, (c_2^*, \text{ctx}^*), \sigma^*) = 1$, and $(\cdot, \cdot, c_2^*, \text{ctx}^*) \notin T_{\text{seen}}$. The first condition follows from the fact that Moderate succeeded, and the second by construction of the abort condition. But this means $((c_2^*, \text{ctx}^*), \sigma^*)$ must be an existential forgery of the MAC. If σ^* were a tag returned from a MAC.Sign query to $\mathcal{B}$'s challenger, then it necessarily would have been recorded in T_{seen} , as the only way $\mathcal{B}$ generates such queries is from within $\mathcal{O}_{\text{SendMsg}}$. $\square$

Lemma C.4. *Suppose that the advantage of any binding adversary $\mathcal{B}$ in breaking the binding of Com is at most $\text{BINDAdv}[\mathcal{B}, \lambda]$. Then for all efficient adversaries $\mathcal{A}$,*

$$|\Pr[\mathbf{H}_3(\mathcal{A})] - \Pr[\mathbf{H}_2(\mathcal{A})]| \leq \text{BINDAdv}[\mathcal{B}, \lambda].$$

Proof. We will show that adversary who can distinguish $\mathbf{H}_3$ from $\mathbf{H}_2$ can be used to build an adversary $\mathcal{B}$ that can break the binding of the commitment scheme. Suppose that there exists such an adversary $\mathcal{A}$. Since the new abort condition is the only difference between these two hybrids, $\mathcal{A}$'s distinguishing advantage is bounded above by the probability that $\mathcal{A}$ triggers the abort condition. Now define an adversary $\mathcal{B}$ in the commitment binding experiment as follows. $\mathcal{B}$ simulates $\mathbf{H}_3$ to $\mathcal{A}$ and monitors if $\mathcal{A}$ has triggered the abort condition. If so, it has constructed an m^* and $\text{report}^* = (r^*, c_2^*, \text{ctx}^*, \sigma^*)$ such that $(m^*, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$ and, because Moderate succeeded, $\text{Com.Open}(c_2^*, m^*, r^*) = 1$.

Since the abort condition added in $\mathbf{H}_2$ must not trigger in order to reach the abort condition added in $\mathbf{H}_3$, we know that $(m, r, c_2^*, \text{ctx}^*) \in T_{\text{seen}}$ for some m and r . Moreover, the new abort condition requires that $(m^*, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$, so we must have $m^* \neq m$. $\mathcal{B}$ can now submit $(m^*, r^*, c_2^*), (m, r, c_2^*)$ as its commitment binding counterexample. Since $\mathcal{B}$ has runtime complexity identical to $\mathcal{A}$, this completes the proof. $\square$

Theorem C.5 (Basic Scheme Deniability). *Assuming that the public key encryption scheme PKE is CPA-secure, our third-party franking scheme Π_{Basic} , described in Construction 5.1, satisfies deniability (Definition B.3).*

Proof. Define the forgery algorithms as follows.

```

Forge(pks, target,  $\text{sk}_p, k_{\text{reg}}, \text{ks}, \text{sk}_{m,\text{target}}, \text{sk}_{m,p,\text{target}}, m, \text{ctx})$ 
```

```

 $r \xleftarrow{\mathcal{R}} \mathcal{R}$ 
 $c_2 \leftarrow \text{Com.Commit}(m, r)$ 
 $\text{sk}_{m,p,\text{target}}, \cdot \leftarrow \text{ks}[\text{target}]$ 
 $\sigma_{\text{pt}} \leftarrow 0 \quad \sigma_{\text{pt}} \leftarrow \text{MAC.Sign}(\text{sk}_{m,p,\text{target}}, (c_2, \text{ctx}))$ 
 $\sigma \leftarrow \text{Enc}(\text{pks}[\text{target}], \sigma_{\text{pt}})$ 
 $\text{report} \leftarrow (r, c_2, \text{ctx}, \sigma)$ 
return report
```

Now note that the only information the adversary receives in all DENY experiments is report_b , so it suffices to show that $\{\text{report}_0\}$ is distributed computationally indistinguishably from $\{\text{report}_1\}$. In both experiments, r is selected uniformly at random, ctx is selected by the adversary, and c_2 is a deterministic function of r and the adversary's input m . So the only piece computed differently across the two experiments is σ' . We address U-DENY and {PC, MC}-DENY separately.

Universal Deniability. We will show that the universal deniability experiments U-DENY $[\mathcal{A}, \Pi_{\text{Basic}}, N, \text{Forge}, 0, \lambda]$ and U-DENY $[\mathcal{A}, \Pi_{\text{Basic}}, N, \text{Forge}, 1, \lambda]$ are indistinguishable by a standard reduction to the CPA security of PKE. Observe that in these experiments, the adversary does not have access to $\text{sk}_{m,\text{target}}$. Thus the reduction $\mathcal{B}$ works by playing the role of the adversary in the CPA security game and the challenger to adversary $\mathcal{A}$. For each query to $\mathcal{O}_{\text{SendMsg}}$, it sets $m_0 = 0$ and $m_1 = \text{MAC.Sign}(\text{sk}_{m,p,\text{target}}, (c_2, \text{ctx}))$. Thus if the CPA challenger has $b = 0$, the reduction is giving $\mathcal{A}$ a perfect simulation of U-DENY $[\mathcal{A}, \Pi_{\text{Basic}}, N, \text{Forge}, 0, \lambda]$. On the other hand, if the CPA challenger has $b = 1$, the reduction is giving $\mathcal{A}$ a perfect simulation of U-DENY $[\mathcal{A}, \Pi_{\text{Basic}}, N, \text{Forge}, 1, \lambda]$. The reduction $\mathcal{B}$ passes on the output of $\mathcal{A}$ as its own output, winning the CPA security game with the same advantage that $\mathcal{A}$ distinguishes between the two universal deniability experiments.

Platform and Moderator Compromise. Note that the experiments for PC-DENY and MC-DENY offer identical views to the adversary, and only differ in the function signature of Forge. But because the code running inside Forge is identical in both cases, for convenience we discuss both at the same time. For these experiments, σ' is computed as the encryption of a MAC of c_2 and ctx in both report_0 and report_1 . Therefore the full view of the adversary is distributed identically in both experiments.

Since in all three deniability experiments, report_0 is distributed indistinguishably from report_1 , our basic scheme satisfies deniability. $\square$

Theorem C.6 (Basic Scheme Subpoena Safety). *Assuming that the public key encryption scheme PKE is CPA-secure, our third-party franking scheme Π_{Basic} , described in Construction 5.1, satisfies subpoena safety (Definition B.4).*

Proof (sketch). The proof of this theorem is almost identical to that of universal deniability, so we omit a full proof. The subpoena safety forger needs to forge the tuple $(c_1, c_2, \sigma, \text{st})$. Our Forge algorithm computes σ and c_2 in the same way as the deniability forger. It has access to k as an input, so it computes $c_1 \leftarrow \text{Enc}(k, (m, r))$ where r is the randomness used to produce c_2 . It also sets $\text{st} = (\text{ctx}, \text{target})$ using its inputs. Thus everything output by the forger in received_1 is distributed identically to the corresponding values in received_0 , except for σ . Thus the indistinguishability of received_0 and received_1 is a reduction to the CPA security of PKE.Enc, identical to the one used in the proof of universal deniability in Theorem C.5.

□

Theorem C.7 (Basic Scheme Moderator Binding). *Assuming that MAC satisfies existential unforgeability, our third-party franking scheme Π_{Basic} , described in Construction 5.1, satisfies moderator binding (Definition B.5).*

Specifically, for any adversary $\mathcal{A}$ attacking the unforgeability of our scheme Π_{Basic} , for all $N, Q \in \mathbb{N}$, and for any security parameter λ , there exists an adversary $\mathcal{B}$ attacking the unforgeability of MAC such that

$$\text{MODBINDAdv}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q, \lambda] \leq \text{MACAdv}[\mathcal{B}, \lambda].$$

Proof. We will show how to use an adversary $\mathcal{A}$ who breaks moderator binding to build an adversary $\mathcal{B}$ who breaks the unforgeability of MAC.

To start, $\mathcal{B}$ first simulates the experiment $\text{MODBIND}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q, \lambda]$, with the exception of one change: for the moderator target, it replaces that moderator's MAC.Sign and MAC.Verify calls with queries to its own MAC challenger. At any point, if the line $\text{win} \leftarrow 1$ is reached immediately after a call to Moderate with report $(r^*, c_2^*, \text{ctx}^*, \sigma^*)$, $\mathcal{B}$ submits $((c_2^*, \text{ctx}^*), \sigma^*)$ as its MAC forgery. From $\mathcal{A}$'s perspective, $\mathcal{B}$ presents precisely the experiment $\text{MODBIND}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q, \lambda]$.

We now show that $\mathcal{B}$ forges a MAC with the same advantage that $\mathcal{A}$ wins in the moderator binding experiment. Observe that since the win condition has the condition $i \neq \text{target}$, the adversary cannot win on a call to $\mathcal{O}_{\text{Mod}}$ where $i = \text{target}$. This implies that the table T_{seen} is always a superset of the values of ctx that the reduction $\mathcal{B}$ has used in queries to the MAC challenger. Thus if Moderate succeeds when $i \neq \text{target}$, it must be with a value of ctx^* that $\mathcal{B}$ never sent to the MAC challenger. But since Moderate requires a successful call to MAC.Verify this means setting $\text{win} \leftarrow 1$ requires producing values $((c_2^*, \text{ctx}^*), \sigma^*)$ where ctx^* was not previously included in a MAC signing query for the target moderator. This is exactly the condition needed to forge a MAC, so $\mathcal{B}$ will have a successful forgery whenever $\mathcal{A}$ wins the experiment. □

Appendix D.

Proof of Security for Mod-Priv-1

Theorem D.1 (Mod-Priv-1 Accountability). *Assuming that MAC satisfies perfect correctness, our third-party franking scheme Π_{MP1} , described in Construction 6.1 (Mod-Priv-1), satisfies accountability (Definition B.1).*

Specifically, for all efficient adversaries $\mathcal{A}$, all $N \in \mathbb{N}$, and any security parameter λ , our scheme Π_{MP1} satisfies

$$\text{ACCTAdv}[\mathcal{A}, \Pi_{\text{MP1}}, N, \lambda] = 0.$$

Proof. This proof is almost identical to that of the basic scheme, except that the correctness of the encryption scheme PKE is replaced by the construction of the re-encryption scheme RE shown in Figure 3.

For the sake of contradiction, suppose that the adversary reaches the line setting $\text{win} \leftarrow 1$. Then we must have

$(m, i, \text{report}) \neq \perp$, and either $\text{res} \neq \text{ctx}$ or $\text{res} = \perp$. Since the experiment honestly executes Read , ReportGen , and Moderate in $\mathcal{O}_{\text{SendMsg}}$, the constraint $(m, i, \text{report}) \neq \perp$ implies that $\text{Com.Open}(c_2, m, r) = 1$. Otherwise, Read would have failed and output $\perp$. Therefore, since commitment opening is a deterministic process, it must also hold that $\text{Com.Open}(c_2, m, r) = 1$ when this is checked in Moderate . This means there are two ways for the constraint $\text{res} \neq \text{ctx} \vee \text{res} = \perp$ to be met:

- 1) $\text{MAC.Verify}(\text{sk}_{m,p}, (c_2, r', \text{ctx}), \sigma_{\text{pt}}) = 0$, or
- 2) $\text{res} \neq \text{ctx}$.

Note that (1) is not possible: since σ_i and c_3 are computed honestly in Process , this follows by construction of σ_i , construction of RE, and correctness of MAC. In particular, the succession of honestly executed calls to RE.Enc , RE.ReEnc , and RE.Dec result in correct recovery of the plaintext r' , meaning the MAC σ is being checked on the same inputs over which it was computed. Similarly, (2) is not possible since, by the construction of our scheme, the ctx included inside report is the same one used to compute σ inside Process . Therefore it is not possible to reach the line $\text{win} \leftarrow 1$, and our scheme has perfect accountability. □

Theorem D.2 (Mod-Priv-1 Unforgeability). *Assuming that MAC satisfies existential unforgeability and the commitment scheme Com satisfies binding, our third-party franking scheme Π_{MP1} (Mod-Priv-1), described in Construction 6.1, satisfies unforgeability (Definition B.2).*

Specifically, for any adversary $\mathcal{A}$ attacking the unforgeability of our scheme Π_{MP1} , for all $N, Q_S, Q_F \in \mathbb{N}$, and for any security parameter λ , there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ attacking the unforgeability and binding of MAC and Com , respectively, such that

$$\begin{aligned} \text{FORGAdv}[\mathcal{A}, \Pi_{\text{MP1}}, N, Q_S, Q_F, \lambda] \\ \leq \text{MACAdv}[\mathcal{B}, \lambda] + \text{BINDAdv}[\mathcal{C}, \lambda]. \end{aligned}$$

Proof. Our proof proceeds as a series of hybrids. The proof of this theorem is almost identical to that of the basic scheme, except for the additional inclusion of r' in $\mathbf{H}_1$ and its appearance as part of the logic of subsequent hybrids.

- $\mathbf{H}_0$ This is the unforgeability experiment $\text{FORG}[\mathcal{A}, \Pi_{\text{Basic}}, N, Q_S, Q_F, \lambda]$.
- $\mathbf{H}_1$ This hybrid is identical to the previous one, except we augment the experiment's table T_{seen} to record tuples $(m, r, c_2, r', \text{ctx})$, and now check whether $(m^*, \cdot, \cdot, \cdot, \text{ctx}^*) \in T_{\text{seen}}$ in the ForgeMsg oracle. This hybrid is indistinguishable from the previous one, as it doesn't result in any change in the adversary's view.
- $\mathbf{H}_2$ This hybrid is identical to the previous one, except we add an abort condition: the experiment aborts and outputs 0 if, in a call to $\mathcal{O}_{\text{ForgeMsg}}$, $\text{ctx}^* \neq \perp$ (in other words, Moderate succeeded), and $(\cdot, \cdot, c_2^*, r'^*, \text{ctx}^*) \notin T_{\text{seen}}$, where c_2^* is included in report^* and r'^* is the plaintext decrypted from c_3^* , which is included in report^* . By the unforgeability of the MAC scheme and Lemma D.3, this hybrid is indistinguishable from the preceding one.

- **H₃** This hybrid is identical to the previous one, except we add a new abort condition: the experiment aborts and outputs 0 if, in a call to $\mathcal{O}_{\text{ForgeMsg}}$, $\text{ctx}^* \neq \perp$, and $(m^*, \cdot, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$. By the binding of the commitment scheme and Lemma D.4, this hybrid is indistinguishable from the preceding one.

We now show that in **H₃**, the adversary has unforgeability advantage 0. For the sake of contradiction, suppose that, at some point in the experiment, $\mathcal{O}_{\text{ForgeMsg}}$ outputs 1. This means that both of the following are true:

- $\text{Moderate}(\text{sk}_{m,\text{target}}, \text{sk}_{m,p,\text{target}}, m^*, \text{report}^*) \neq \perp$, and
- $(m^*, \cdot, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$.

But if both of these are satisfied, then this is a contradiction, as the experiment would have aborted and output 0, by the abort condition added in **H₃**. Since $\mathcal{A}$ has no advantage, the proof of the theorem follows from the proofs of the lemmas and the triangle inequality. $\square$

Lemma D.3. *Suppose that the advantage of any MAC adversary $\mathcal{B}$ in breaking the existential unforgeability of MAC is at most $\text{MACAdv}[\mathcal{B}, \lambda]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_2(\mathcal{A})] - \Pr[\mathbf{H}_1(\mathcal{A})] \right| \leq \text{MACAdv}[\mathcal{B}, \lambda].$$

We omit the proof of this lemma as it is exactly analogous to the proof of Lemma C.3, with the contents of the table T_{seen} adjusted to include r' .

Lemma D.4. *Suppose that the advantage of any binding adversary $\mathcal{B}$ in breaking the binding of Com is at most $\text{BINDAdv}[\mathcal{B}, \lambda]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_3(\mathcal{A})] - \Pr[\mathbf{H}_2(\mathcal{A})] \right| \leq \text{BINDAdv}[\mathcal{B}, \lambda].$$

We omit the proof of this lemma as it is exactly analogous to the proof of Lemma C.4, with the contents of the table T_{seen} and report^* adjusted to include an extra entry that does not play an important rule in the proof.

Theorem D.5 (Mod-Priv-1 Deniability). *Assuming that DDH is hard in $\mathbb{G}$, and modeling MAC.Sign as a random oracle, our third-party franking scheme Π_{MP1} (Mod-Priv-1), described in Construction 6.1, satisfies deniability (Definition B.3).*

Proof (sketch). Define the forgery algorithms as follows.

$\text{Forge}(\text{pks}, \text{target}, \text{sk}_p, k_{\text{reg}}, \text{ks}, \text{sk}_{m,\text{target}}, \text{sk}_{m,p,\text{target}}, m, \text{ctx})$

$r, r' \xleftarrow{\mathcal{R}} \mathcal{R} \times \mathbb{G}$

$c_2 \leftarrow \text{Com.Commit}(m, r)$

$\cdot, \text{pk}_{m,\text{target},2} \leftarrow \text{pks}[\text{target}]$

$c'_3 \leftarrow \text{RE.Enc}(\text{pk}_{m,\text{target},2}, r')$

$\text{sk}_{m,p,\text{target}}, \cdot \leftarrow \text{ks}[\text{target}]$

$\sigma \xleftarrow{\mathcal{T}} \sigma \leftarrow \text{MAC.Sign}(\text{sk}_{m,p,\text{target}}, (c_2, r', \text{ctx}))$

$\text{report} \leftarrow (r, c_2, c'_3, \text{ctx}, \sigma)$

return report

Note that the only information the adversary receives in the deniability experiments is report_b , so it suffices to show that $\{\text{report}_0\}$ is distributed computationally indistinguishably from $\{\text{report}_1\}$. In both experiments, r is selected uniformly at random, ctx is selected by the adversary, c_2 is a deterministic function of r and the adversary's input m , and c'_3 is the encryption of a randomly selected r' . So the only piece computed differently across the two experiments is σ . We address U-DENY and {PC, MC}-DENY separately.

Universal Deniability. The proof that σ is indistinguishable between report_0 and report_1 first uses the hardness of DDH in G to replace c'_3 with an encryption of a new random value r'' and then invokes the fact that MAC.Sign is modeled as a random oracle to demonstrate that an adversary cannot distinguish between a real MAC tag in report_0 and a random one in report_1 unless it guesses the value of r' , which occurs with at most negligible probability. We do not formalize this argument here because it is very similar to the proof of subpoena safety for this scheme, and the proof is presented formally there.

Platform and Moderator Compromise. Note that the experiments for PC-DENY and MC-DENY offer identical views to the adversary, and only differ in the function signature of Forge. But because the code running inside Forge is identical in both cases, for convenience we discuss both at the same time. For these experiments, σ is computed as the MAC of c_2, r' , and ctx in both report_0 and report_1 . Therefore the full view of the adversary is distributed identically in both experiments.

Since in all three deniability experiments, report_0 is distributed indistinguishably from report_1 , Mod-Priv-1 satisfies deniability. $\square$

Theorem D.6 (Mod-Priv-1 Subpoena Safety). *Assuming that DDH is hard in $\mathbb{G}$, and modeling MAC.Sign as a random oracle, our third-party franking scheme Π_{MP1} (Mod-Priv-1), described in Construction 6.1, satisfies subpoena safety (Definition B.4).*

Specifically, for all efficient adversaries $\mathcal{A}$ who make at most Q queries to the $\mathcal{O}_{\text{SendMsg}}$ oracle and Q_{RO} queries to the random oracle MAC.Sign, all $N, Q, Q_{\text{RO}} \in \mathbb{N}$, and any security parameter λ , there exists a DDH adversary $\mathcal{B}$ for $\mathbb{G}$ such that

$$\begin{aligned} \text{SUBPOENAAAdv}[\mathcal{A}, \Pi_{\text{MP1}}, N, Q, \lambda] \\ \leq 2 \cdot \text{DDHAdv}[\mathcal{B}, \mathbb{G}] + 2NQ_{\text{RO}}/q. \end{aligned}$$

Proof. Define the forgery algorithm as follows, where $\mathcal{T}$ is the output space of MAC.Sign.

Forge(pks, target, k, m, ctx)

```

pk1, pk2, k1→2 ← pks[target]
s, epk ← RE.MaskKey(pk1)
ke→2 ← RE.ReKeyGen(s, k1→2)
r, r'  $\xleftarrow{\mathbb{R}}$   $\mathcal{R} \times \mathbb{G}$ 
c1 ← Enc(k, (m, target, r, ke→2))
c2 ← Com.Commit(m, r)
c3 ← RE.Enc(epk, r')
st ← (c3, epk, ctx)
σ  $\xleftarrow{\mathbb{R}}$   $\mathcal{T}^N$ 
return (c1, c2, σ, st)

```

Observe that the only difference between the output received₁ of Forge and the values of (c₁, c₂, σ, st) output in received₀ is the computation of σ. We will show that these values are computationally indistinguishable through a series of hybrids.

- **H₀** : This is the subpoena safety experiment SUBPOENA[$\mathcal{A}, \Pi_{\text{MP1}}, N, Q, 0, \lambda$].
- **H₁** : This hybrid is identical to the preceding one, except we replace the value pk^r in RE.Enc with a uniformly random element of $\mathbb{G}$.
By the hardness of DDH in $\mathbb{G}$ and Lemma D.7, this hybrid is indistinguishable from the preceding one.
- **H₂** : This hybrid is identical to the preceding one, except we replace the value r' encrypted in c₃ with a uniformly random r''.
This hybrid is distributed identically to the preceding one because the value r'' is multiplied by g^r for an independently random r not used elsewhere (this variable name is overloaded in RE and is distinct from the r used in Send).
 - **H₃** : This hybrid is identical to the preceding one, except the experiment aborts and outputs 0 if the adversary ever queries the random oracle MAC.Sign on input r' (in the appropriate position) for any value of r' sampled during any call to $\mathcal{O}_{\text{SendMsg}}$.
This hybrid is indistinguishable from the previous one as long as the new abort condition is not triggered. But since r' appears nowhere in the view of the adversary except as an input to MAC.Sign, the probability that the adversary makes a query that triggers the abort condition is 1/q for each call to MAC.Sign for each value of r', since r' is chosen uniformly at random from the group $\mathbb{G}$ of order q. Each call to $\mathcal{O}_{\text{SendMsg}}$ selects a random r' for each of the N moderators, so by the union bound the overall probability of triggering the abort condition is at most NQQ_{RO}/q .
 - **H₄** : This hybrid is identical to the preceding one, except we compute the tags $\sigma = \sigma_1, \dots, \sigma_N$ as $\sigma \xleftarrow{\mathbb{R}} \mathcal{T}^N$. This hybrid is distributed identically to the preceding one because the values of σ already appear uniformly random to an adversary who does not trigger the abort condition added in **H₃**, as MAC.Sign is modeled as a random oracle.

- **H₅** : This hybrid undoes the change made in **H₃**, removing the abort condition added in that hybrid. It is statistically indistinguishable from the preceding hybrid by the same reasoning provided above, so an adversary can distinguish with probability at most NQQ_{RO}/q .
- **H₆** : This hybrid undoes the change made in **H₂**, encrypting r' in c₃ instead of a new random r''. This hybrid is distributed identically to the preceding one by the same reasoning provided in the discussion of **H₂**.
- **H₇** : This hybrid undoes the change made in **H₁**, using the value of pk^r in RE.Enc as specified in the original scheme.

By the hardness of DDH in $\mathbb{G}$ and Lemma D.8, this hybrid is indistinguishable from the preceding one.

Observe that hybrid **H₇** is identical to the subpoena safety experiment $\Pr[\text{SUBPOENA}[\mathcal{A}, \Pi_{\text{MP1}}, N, Q, 1, \lambda]]$. The proof of the theorem thus follows from the indistinguishability of each successive pair of hybrids and the triangle inequality. $\square$

Lemma D.7. Suppose that the advantage of any adversary $\mathcal{B}$ in breaking DDH in $\mathbb{G}$ is at most $\text{DDHAdv}[\mathbb{G}, \mathcal{B}]$. Then for all efficient adversaries $\mathcal{A}$,

$$\left| \Pr[\mathbf{H}_1(\mathcal{A})] - \Pr[\mathbf{H}_0(\mathcal{A})] \right| \leq \text{DDHAdv}[\mathbb{G}, \mathcal{B}].$$

Proof. We show how to build an adversary $\mathcal{B}$ who uses $\mathcal{A}$ to win the DDH experiment with the same advantage that $\mathcal{A}$ distinguishes between the two hybrids. Adversary $\mathcal{B}$ plays the role of the adversary in the DDH security experiment, receiving values (X, Y, Z) from the DDH challenger, while simulating the subpoena safety challenger to $\mathcal{A}$.

Adversary $\mathcal{B}$ plays the role of the **H₁** challenger exactly, except it makes the following changes when running SetupMod for moderator $i = \text{target}$:

- Set $\text{pk}_1 \leftarrow X$.
- Set $k_{1 \rightarrow 2} \xleftarrow{\mathbb{R}} \mathbb{Z}_q$.
- Set $\text{pk}_2 \leftarrow \text{pk}_1^{k_{1 \rightarrow 2}}$.
- Do not set a value for sk_m . Observe that this value is not given to $\mathcal{A}$ and is only used in Moderate, which is never called in this experiment.

Next, whenever $\mathcal{A}$ makes a call to the $\mathcal{O}_{\text{SendMsg}}$ oracle, $\mathcal{B}$ plays the role of the **H₁** challenger exactly, except for in calls to RE.Enc it samples a random $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_q$ and returns $(Y^\alpha \cdot r', Z^{\alpha s})$. At the end of the experiment, $\mathcal{B}$ passes on $\mathcal{A}$'s output b' as its own distinguishing bit in the DDH experiment.

We argue that $\mathcal{B}$ successfully distinguishes between $X = g^x, Y = g^y, Z = g^{xy}$ and $X = g^x, Y = g^y, Z = g^z$ (for $x, y, z \xleftarrow{\mathbb{R}} \mathbb{Z}_q$) with the same advantage that $\mathcal{A}$ distinguishes between **H₀** and **H₁**. Observe that the value of pk_1 is a random group element $X = g^x$, where $\text{sk}_1 = x$, and since $\text{pk}_2 \leftarrow \text{pk}_1^{k_{1 \rightarrow 2}}$, this means that, by definition, if $\text{pk}_2 = g^{\text{sk}_2}$, then $k_{1 \rightarrow 2} = \text{sk}_2/x$.

If $Z = g^{xy}$, then $\mathcal{B}$ has provided $\mathcal{A}$ a perfect simulation of **H₀**, because setting the first part of the ciphertext to Y^α

just means choosing encryption randomness $y \cdot \alpha$, and since $\text{epk} = \text{pk}_1^s = g^{xs}$, this means that

$$Z^{\alpha s} = (g^{xy})^{\alpha s} = \text{pk}_1^{y\alpha s} = \text{epk}^{y\alpha},$$

which is exactly how encryption is computed in $\mathbf{H}_0$. On the other hand, if $Z = g^z$, then $\mathcal{B}$ has provided $\mathcal{A}$ a perfect simulation of $\mathbf{H}_1$ because the second part of the ciphertext is set to $Z^{\alpha s} = (g^z)^{\alpha s}$, where z is a uniformly random exponent. Thus $\mathcal{B}$ must prevail in the DDH experiment with the same advantage that $\mathcal{A}$ distinguishes between the two hybrids. $\square$

Lemma D.8. *Suppose that the advantage of any adversary $\mathcal{B}$ in breaking DDH in $\mathbb{G}$ is at most $\text{DDHAdv}[\mathcal{B}, \mathbb{G}]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_7(\mathcal{A})] - \Pr[\mathbf{H}_6(\mathcal{A})] \right| \leq \text{DDHAdv}[\mathcal{B}, \mathbb{G}].$$

We omit a proof of this lemma, as it is analogous to the proof of Lemma D.7.

Theorem D.9 (Mod-Priv-1 Moderator Privacy). *Assuming that the underlying symmetric encryption scheme is CPA secure, our third-party franking scheme Π_{MP1} (Mod-Priv-1), described in Construction 6.1, satisfies moderator privacy (Definition B.5).*

Specifically, for any adversary $\mathcal{A}$ attacking the moderator privacy of our scheme Π_{MP1} , for all $N, U \in \mathbb{N}$, and for any security parameter λ , there exists an adversary $\mathcal{B}$ attacking the CPA security of Enc, such that

$$\text{MODPRIVAdv}[\mathcal{A}, \Pi_{\text{MP1}}, N, U, \lambda] \leq 2U \cdot \text{CPAAdv}[\mathcal{B}, \lambda].$$

Proof. This proof proceeds as a series of hybrids.

- $\mathbf{H}_0$: This is the moderator privacy experiment $\text{MODPRIV}[\mathcal{A}, \Pi_{\text{MP1}}, N, U, 0, \lambda]$.
- $\mathbf{H}_1 \dots \mathbf{H}_U$: In this sequence of U hybrids, each hybrid $\mathbf{H}_j$ is identical to the preceding one, except that in each query to $\mathcal{O}_{\text{SendMsg}}(j, \cdot, \cdot, \cdot, \cdot)$ or $\mathcal{O}_{\text{Mod}}(j, \cdot, \cdot, \cdot)$, instead of computing c_1 as the encryption of $(m, i_0, r, k_{e \rightarrow 2})$ or $(m, i, r, k_{e \rightarrow 2})$, the experiment instead computes the encryption of $(m, 0, r, 0)$. Here, 0 is used as a shorthand to represent a vector of zeros of lengths equal to the replaced values. When decrypting these ciphertexts, the experiment uses a lookup table to recover the intended values of i and $e_{k \rightarrow 2}$ and proceeds using those as if they were the outputs from the decryption. Each of these hybrids is indistinguishable from the preceding one by the CPA security of the symmetric encryption scheme. We omit the standard indistinguishability proof for brevity.
- $\mathbf{H}_{U+1}$: This hybrid is identical to the previous one, except that in each query to $\mathcal{O}_{\text{SendMsg}}$, the experiment switches from using i_0 to using i_1 in calls to $\mathcal{O}_{\text{SendMsg}}$. This hybrid is identical to the preceding one. The only changes are in the choice of $k_{1 \rightarrow 2}$ (and therefore $k_{e \rightarrow 2}$), which is not included in the view of the adversary, and in the choice of epk , which switches from $\text{epk} \leftarrow \text{pk}_{m, i_0}^s$ to $\text{epk} \leftarrow \text{pk}_{m, i_1}^s$. But $s \xleftarrow{\mathbb{R}} \mathbb{Z}_q$ in

both cases, where s is not part of the adversary's view (it is only used as an input when computing $k_{e \rightarrow 2}$), so epk appears to be a uniformly random element of $\mathbb{G}$ in either case.

- $\mathbf{H}_{U+1} \dots \mathbf{H}_{2U+1}$: This series of hybrids undoes the changes made in $\mathbf{H}_1 \dots \mathbf{H}_U$, replacing the 0 values in the encryptions with the actual values intended to be decrypted.

Each of these hybrids is indistinguishable from the preceding one by the CPA security of the symmetric encryption scheme. As before, we omit the standard indistinguishability proof for brevity.

Now note that $\mathbf{H}_{2U+1}$ is identical to the experiment $\text{MODPRIV}[\mathcal{A}, \Pi_{\text{MP1}}, N, U, 1, \lambda]$. The theorem follows from the indistinguishability of adjacent pairs of hybrids and the triangle inequality. $\square$

Theorem D.10 (Mod-Priv-1 Moderator Binding). *Assuming that DDH is hard in $\mathbb{G}$ and modeling MAC.Sign as a random oracle with output space $\mathcal{T}$, our third-party franking scheme Π_{MP1} (Mod-Priv-1), described in Construction 6.1, satisfies moderator binding (Definition B.6).*

Specifically, for any adversary $\mathcal{A}$ attacking the moderator binding of our scheme Π_{MP1} who makes at most Q queries to the oracle $\mathcal{O}_{\text{Mod}}$ and Q_{RO} queries to the random oracle, for all $N, Q, Q_{\text{RO}} \in \mathbb{N}$, and for any security parameter λ , there exists an adversary $\mathcal{B}$ attacking DDH in $\mathbb{G}$ such that

$$\begin{aligned} \text{MODBINDAdv}[\mathcal{A}, \Pi_{\text{MP1}}, N, Q, \lambda] \\ \leq \text{DDHAdv}[\mathcal{B}, \mathbb{G}] + QQ_{\text{RO}}/q + Q^2/q + Q/|\mathcal{T}|. \end{aligned}$$

Proof. We refer to the values of σ_{target} , r' , and i present during the j th call to $\mathcal{O}_{\text{Mod}}$ as $\sigma_{\text{target}, j}$, r'_j , and i_j , respectively. The proof begins with a series of hybrids.

- $\mathbf{H}_0$: This is the moderator binding experiment $\text{MODBIND}[\mathcal{A}, \Pi_{\text{MP1}}, N, Q, \lambda]$.
- $\mathbf{H}_1$: This hybrid is identical to the preceding one, except in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$, we replace the value pk^r in RE.Enc with a uniformly random element of $\mathbb{G}$.

By the hardness of DDH in $\mathbb{G}$ and Lemma D.11, this hybrid is indistinguishable from the preceding one.

- $\mathbf{H}_2$: This hybrid adds an abort condition to the experiment. If the adversary ever queries the random oracle at an input $\cdot, (\cdot, r', \cdot)$ that matches a value of r' chosen during any call to $\mathcal{O}_{\text{Mod}}$ when $i \neq \text{target}$, the experiment aborts.

This hybrid is statistically indistinguishable from the preceding one. Since r' never appears in the adversary's view in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$, the probability of this abort condition being triggered is $1/q$ times the number of random oracle queries and the number of values of r' sampled during the experiment. Thus the overall probability is upper bounded by QQ_{RO}/q .

- $\mathbf{H}_3$: This hybrid adds an abort condition to the experiment. The experiment aborts if the adversary ever produces a value of c'_3 in its generated report such that

$\text{RE.Dec}(\text{sk}_{m,\text{target}}, c'_3) = r'$ for any value of r' sampled in a call to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$.

This hybrid is statistically indistinguishable from the preceding one. Since r' never appears in the adversary's view in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$, the probability of this abort condition being triggered is $1/q$ times the number of values of c'_3 produced by the experiment and the number of values of r' sampled during the experiment when $i \neq \text{target}$. These values increase by at most one per call to $\mathcal{O}_{\text{Mod}}$, so the probability of the abort condition being triggered is at most Q^2/q .

We complete the proof by showing that in $\mathbf{H}_3$, the adversary wins the experiment with at most negligible probability. The theorem follows from this argument, the indistinguishability of successive pairs of hybrids, and the triangle inequality.

We will consider a number of disjoint cases where the adversary reaches the line where $\text{win} \leftarrow 1$.

- 1) The adversary wins in the j th call to $\mathcal{O}_{\text{Mod}}$ with a report that contains $\sigma_{\text{target},j}$.
- 2) The adversary wins in the j th call to $\mathcal{O}_{\text{Mod}}$ with a report that contains $\sigma_{\text{target},j'}$ for some $j' \neq j$.
- 3) The adversary wins in the j th call to $\mathcal{O}_{\text{Mod}}$ with a report that contains σ_{target}^* where $\sigma_{\text{target}}^* \neq \sigma_{\text{target},j}$ for any $j \in \{1, \dots, Q\}$.

Case 1: If $i = \text{target}$, then the adversary cannot win in this call to $\mathcal{O}_{\text{Mod}}$, so it can only win if $i \neq \text{target}$. But then observe that, since the abort conditions must not be triggered for the adversary to win, the value of r'_j is never seen by the adversary, never queried to the random oracle, and never the result of decrypting c'_3 . Thus the winning report must contain a value $r'^* \neq r'_j$ and cause the random oracle to output $\sigma_{\text{target},j}$, which occurs with probability $1/|\mathcal{T}|$.

Case 2: If $i_j \neq \text{target}$, then the analysis of this case is identical to that of case 1 because the adversary will never have seen r'_j , never queried the random oracle at r'_j , and never produced a value of c'_3 that decrypts to r'_j .

If $i_j = \text{target}$, then note that the winning value of ctx must differ from ctx_j , or else the experiment will abort because $\text{ctx}_j \in \text{seen}$. But this means the adversary must produce some $(c_2^*, r'^*, \text{ctx}^*)$ with $\text{ctx}^* \neq \text{ctx}_j$ such that

$$\text{MAC.Sign}(\text{ks}[\text{target}], (c_2^*, r'^*, \text{ctx}^*)) = \sigma_{\text{target},j'}.$$

Since MAC.Sign is modeled as a random oracle and $\text{ks}[\text{target}]$ is not seen by the adversary, the probability that any report from the adversary satisfies this equation is at most $1/|\mathcal{T}|$.

Case 3: The adversary only wins in this setting if it outputs values of σ^* , $(c_2^*, r'^*, \text{ctx}^*)$ such that

$$\text{MAC.Sign}(\text{ks}[\text{target}], (c_2^*, r'^*, \text{ctx}^*)) = \sigma_{\text{target}}^*.$$

But since the adversary does not see $\text{ks}[\text{target}]$, the probability that any choice of the other values satisfies the equation is $1/|\mathcal{T}|$.

We have shown that in all possible cases, the probability that the adversary wins in the j th query to $\mathcal{O}_{\text{Mod}}$ is at most

$1/|\mathcal{T}|$. By union bound over the Q queries to $\mathcal{O}_{\text{Mod}}$, we have that it wins with probability at most $Q/|\mathcal{T}|$. $\square$

Lemma D.11. *Suppose that the advantage of any adversary $\mathcal{B}$ in breaking DDH in $\mathbb{G}$ is at most $\text{DDHAdv}[\mathcal{B}, \mathbb{G}]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_1(\mathcal{A})] - \Pr[\mathbf{H}_0(\mathcal{A})] \right| \leq \text{DDHAdv}[\mathcal{B}, \mathbb{G}].$$

Proof (sketch). The proof of this lemma is very similar to that of Lemma D.7, except there are multiple public keys that need to be simulated instead of just one. We handle this by, for each $i \neq \text{target}$, sampling $\beta \xleftarrow{\mathbb{R}} \mathbb{Z}_q$ and setting $\text{pk}_{m,i} \leftarrow X^\beta$. The values X^β and Z^β are used by $\mathcal{B}$ wherever it would send $\mathcal{A}$ some function of X or Z in that proof.

Observe that although the $\mathcal{O}_{\text{Mod}}$ oracle includes a call to Moderate , which needs to decrypt adversary-supplied ciphertexts, that function is only ever called with the keys of the target moderator, and encryptions of $i = \text{target}$ are not affected by the change between $\mathbf{H}_0$ and $\mathbf{H}_1$. Thus the reduction can generate keys for moderator target itself and encrypt/decrypt those ciphertexts as described in RE. $\square$

Appendix E.

Proof of Security for Mod-Priv-2

Theorem E.1 (Mod-Priv-2 Accountability). *Our third-party franking scheme Π_{MP2} , described in Construction 6.3 (Mod-Priv-2), satisfies accountability (Definition B.1).*

Specifically, for all efficient adversaries $\mathcal{A}$, all $N \in \mathbb{N}$, and any security parameter λ , our scheme Π_{MP2} satisfies

$$\text{ACCTAdv}[\mathcal{A}, \Pi_{\text{MP2}}, N, \lambda] = 0.$$

Proof. This proof is almost identical to that of Mod-Priv-1, except that we replace the argument citing the correctness of the MAC with a bespoke argument for the MAC-like scheme that Mod-Priv-2 uses in place of a generic MAC.

For the sake of contradiction, suppose that the adversary reaches the line setting $\text{win} \leftarrow 1$. Then we must have $(m, i, \text{report}) \neq \perp$, and either $\text{res} \neq \text{ctx}$ or $\text{res} = \perp$. Since the experiment honestly executes Read , ReportGen , and Moderate in $\mathcal{O}_{\text{SendMsg}}$, the constraint $(m, i, \text{report}) \neq \perp$ implies that $\text{Com.Open}(c_2, m, r) = 1$. Otherwise, Read would have failed and output $\perp$. Therefore, since commitment opening is a deterministic process, it must also hold that $\text{Com.Open}(c_2, m, r) = 1$ when this is checked in Moderate . This means there are two ways for the constraint $\text{res} \neq \text{ctx} \vee \text{res} = \perp$ to be met:

- 1) $\sigma' \neq e(H(c_2, r', \text{ctx})^{k_m}, g_2)$, or
- 2) $\text{res} \neq \text{ctx}$.

Observe that (2) is not possible because, by the construction of our scheme, the ctx included inside report is the same one used to compute σ inside Process . Moreover, (1) is not possible either. Since c_3 is computed honestly in Process , the succession of honestly executed calls to RE.Enc , RE.ReEnc , and RE.Dec result in correct recovery

of the plaintext r' . The value of σ' provided to Moderate as part of report is also computed honestly. Thus we have that

$$\begin{aligned}\sigma' &= e(\sigma, \text{pk}_{\text{proc}}) \\ &= e(H(c_2, r', \text{ctx})^{k_p}, g_2^{k_m/k_p}) \\ &= e(H(c_2, r', \text{ctx}), g_2)^{(k_m/k_p)k_p} \\ &= e(H(c_2, r', \text{ctx}), g_2)^{k_m} \\ &= e(H(c_2, r', \text{ctx})^{k_m}, g_2),\end{aligned}$$

which means that condition (1) cannot be satisfied. Since neither of (1) or (2) are possible, we have reached a contradiction, and the adversary cannot reach the line setting $\text{win} \leftarrow 1$. $\square$

Theorem E.2 (Mod-Priv-2 Unforgeability). *Assuming that DDH is hard in $\mathbb{G}$, that BDH is hard in groups for the pairing e , that the commitment scheme Com satisfies binding, and that H is modeled as a random oracle, our third-party franking scheme Π_{MP2} (Mod-Priv-2), described in Construction 6.3, satisfies unforgeability (Definition B.2).*

Specifically, for any adversary $\mathcal{A}$ attacking the unforgeability of our scheme Π_{MP2} who makes at most Q_S queries to the $\mathcal{O}_{\text{SendMsg}}$ oracle, at most Q_F queries to the $\mathcal{O}_{\text{ForgeMsg}}$ oracle, and at most Q_{RO} queries to the random oracle H , for all $N, Q_S, Q_F, Q_{\text{RO}} \in \mathbb{N}$, and for any security parameter λ , there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ attacking the binding of Com and BDH in the groups for the pairing e , respectively, such that

$$\begin{aligned}\text{FORGAdv}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q_S, Q_F, \lambda] \\ \leq \text{BINDAdv}[\mathcal{B}, \lambda] + Q_F \cdot \text{BDHAdv}[\mathcal{C}, e] \\ + Q_S Q_{\text{RO}}/q.\end{aligned}$$

Proof. Our proof proceeds through a series of hybrids.

- **H₀**: This is the unforgeability experiment $\text{FORG}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q_S, Q_F, \lambda]$.
- **H₁**: This hybrid is identical to the previous one, except we augment the experiment's table T_{seen} to record tuples $(m, r, c_2, r', \text{ctx})$, and now check whether $(m^*, \cdot, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$ in the ForgeMsg oracle. During calls to $\mathcal{O}_{\text{ForgeMsg}}$, we also temporarily define the augmented table $T_{\text{seen}}^* \leftarrow T_{\text{seen}} \cup (m^*, r^*, c_2^*, r'^*, \text{ctx}^*)$, which is T_{seen} with one additional entry corresponding to the adversary's current report. This hybrid is indistinguishable from the previous one, as it doesn't result in any change in the adversary's view.
- **H₂**: This hybrid is identical to the preceding one, except we add an abort condition to the experiment. If at any time, one of the tables T_{seen}^* or T_{seen} contains two entries $(m, \cdot, c_2, \cdot, \cdot)$ and $(m', \cdot, c_2, \cdot, \cdot)$ where $m \neq m'$, the experiment aborts and outputs 0.

In Lemma E.3 we bound an adversary's ability to distinguish this hybrid from the preceding one by its

ability break the binding of the commitment scheme Com.

- **H₃**: This hybrid adds an additional abort condition to the experiment. The experiment aborts if the adversary ever queries H at an input $(\cdot, r', \cdot)$ that is later selected as the randomness used in a call to Process in the $\mathcal{O}_{\text{SendMsg}}$ oracle.

This hybrid is statistically indistinguishable from the preceding one. Since the values of r' in Process are chosen uniformly at random from $\mathbb{G}$, choices of r' made after an adversary's query to H are independent of the adversary's query. Thus for each query the adversary makes to H , the probability that the value of r' used in that query later appears in a call to Process in $\mathcal{O}_{\text{SendMsg}}$ is Q_S/q . Taking a union bound over the number of queries made to H yields a probability of $Q_S Q_{\text{RO}}/q$ as an upper bound on the probability of an adversary triggering this abort condition.

- **H₄**: This hybrid is identical to the preceding one, except we add an additional abort condition to the experiment. The experiment aborts and outputs 0 if the experiment reaches the line $\text{win} \leftarrow 1$ during a call to $\mathcal{O}_{\text{ForgeMsg}}$ with values of c_2^*, r^*, ctx^* in report* where $(\cdot, \cdot, c_2^*, r'^*, \text{ctx}^*) \notin T_{\text{seen}}$.

In Lemma E.4 we bound an adversary's ability to distinguish this hybrid from the preceding one by its ability to solve the computation bilinear Diffie-Hellman problem for the pairing groups in e .

From **H₄**, we show that an adversary cannot win the unforgeability experiment, completing the proof. Observe that, by the abort condition added in **H₄**, whenever an adversary wins the experiment, it must do so with values of $c_2^*, r'^*, \text{ctx}^*$ where $(\cdot, \cdot, c_2^*, r'^*, \text{ctx}^*) \in T_{\text{seen}}$. Moreover, by the abort condition added in **H₂**, there can only be one entry $(m^*, \cdot, c_2^*, \cdot, \cdot)$ in T_{seen} for each value of c_2^* . This means that whenever the adversary reaches $\text{win} \leftarrow 1$, there is an entry $(m^*, \cdot, c_2^*, \cdot, \text{ctx}^*) \in T_{\text{seen}}$. But since the experiment only sets $\text{win} \leftarrow 1$ if $(m^*, \cdot, \cdot, \cdot, \text{ctx}^*) \notin T_{\text{seen}}$, this is a contradiction, and the line $\text{win} \leftarrow 1$ can never be reached. Thus the theorem follows from the proofs of the preceding lemmas and the triangle inequality. $\square$

Lemma E.3. *Suppose that the advantage of any binding adversary $\mathcal{B}$ in breaking the binding of Com is at most $\text{BINDAdv}[\mathcal{B}, \lambda]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_2(\mathcal{A})] - \Pr[\mathbf{H}_1(\mathcal{A})] \right| \leq \text{BINDAdv}[\mathcal{B}, \lambda].$$

Proof. We show how to build an adversary $\mathcal{B}$ that uses an adversary $\mathcal{A}$ who can distinguish between **H₁** and **H₂** to break the binding of the commitment Com with the same advantage that $\mathcal{A}$ distinguishes between the two hybrids. Adversary $\mathcal{B}$ simulates the **H₂** experiment for $\mathcal{A}$ while playing the role of the adversary in the commitment binding experiment, using any public parameters given to it by the binding adversary when computing Com.Commit() and Com.Open(). If at any point $\mathcal{A}$ triggers the abort condition added in **H₂**, we argue that $\mathcal{B}$ can break the binding of Com.

Observe that the only difference between the two hybrids is the abort condition added in $\mathbf{H}_2$, so an adversary's ability to distinguish between the hybrids is upper bounded by the probability that it triggers the abort condition. But if this happens, then, by the definition of the abort condition, the adversary has $(m, r, c_2, \cdot, \cdot)$ and $(m', r', c_2, \cdot, \cdot)$ where $m \neq m'$ but where Moderate has computed $\text{Com.Open}(c_2, m, r) = 1$ and $\text{Com.Open}(c_2, m', r') = 1$. This means that $c_2, (m, r), (m', r')$ constitute a commitment and two different messages to which that commitment can be opened, so $\mathcal{B}$ submits these values to the binding challenger and wins the commitment binding security experiment. $\square$

Lemma E.4. *Suppose that the advantage of any adversary $\mathcal{B}$ in breaking the computational bilinear Diffie Hellman problem (BDH) in the groups for pairing e is at most $\text{BDHAdv}[\mathcal{B}, e]$. Then for all efficient adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{H}_4(\mathcal{A})] - \Pr[\mathbf{H}_3(\mathcal{A})] \right| \leq Q_F \cdot \text{BDHAdv}[\mathcal{B}, e].$$

Proof. Observe that the advantage of any adversary $\mathcal{A}$ in distinguishing between the two hybrids is upper bounded by the probability that the adversary triggers the abort condition added in $\mathbf{H}_4$, as the adversary's view in the two experiments is otherwise identical. We show how to build a BDH adversary $\mathcal{B}$ who uses $\mathcal{A}$ and wins the BDH experiment with probability $1/Q_F$ whenever $\mathcal{A}$ triggers the abort criterion.

Adversary $\mathcal{B}$ plays the role of the BDH adversary while simulating the $\mathbf{H}_4$ challenger to $\mathcal{A}$. Recall that in our variant of BDH, the adversary $\mathcal{B}$ receives $g_1, g_2, X_1 = g_1^x, X_2 = g_2^{1/x}, Y_1 = g_1^y, Y_2 = g_2^{z/x}$ for $x, y, z \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and is tasked with computing a value $Z = e(g_1, g_2)^{yz}$.

We have $\mathcal{B}$ set $k_{\text{reg}} \leftarrow X_2$ and $\text{pk}_{\text{proc, target}} \leftarrow Y_2$. This has the effect of implicitly setting $k_p = x$ and $k_m = z$ for the target moderator. When computing the value of σ in calls to $\mathcal{O}_{\text{SendMsg}}$, adversary $\mathcal{B}$ samples a random $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and sets $\sigma \leftarrow X_1^\alpha$. This implicitly sets $H(c_2, r', \text{ctx})^{k_p} = g_1^{x\alpha}$ and $H(c_2, r', \text{ctx}) = g_1^\alpha$, which is what $\mathcal{B}$ sends to $\mathcal{A}$ if it queries the random oracle at (c_2, r', ctx) later in the experiment.

When the adversary $\mathcal{A}$ makes calls to the random oracle for a fresh input (c_2, r', ctx) outside of $\mathcal{O}_{\text{SendMsg}}$, $\mathcal{B}$ samples a fresh random $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and returns the value Y_1^α . The abort condition added in $\mathbf{H}_3$ ensures that an output to H assigned in this way never needs to be reassigned in a call to Process in $\mathcal{O}_{\text{SendMsg}}$.

The adversary $\mathcal{B}$ selects a query index $j \xleftarrow{\mathbb{R}} \{0, \dots, Q_F - 1\}$ as its guess of the first call to $\mathcal{O}_{\text{ForgeMsg}}$ that results in reaching the line $\text{win} \leftarrow 1$. For queries prior to j where $i = \text{target}$, it always outputs $\perp$ from Moderate. On query j , adversary $\mathcal{B}$ submits $\sigma'^{1/\alpha}$ as its guess of Z in the BDH experiment, where σ' is the value submitted by $\mathcal{A}$ in report and α is the exponent chosen to make $H(c_2, r', \text{ctx}) \leftarrow Y_1^\alpha$ for the values of (c_2, r', ctx) in $\mathcal{A}$'s report. We will show below that $\mathcal{B}$ will always know this α .

Observe that if $\mathcal{B}$ guesses the correct value of j , it provides $\mathcal{A}$ with a perfect simulation of the $\mathbf{H}_4$ security

experiment where $k_p = x$, $k_m = z$, and each evaluation of H appears independently random. Since $\mathcal{B}$ has guessed correctly, the output win of $\mathcal{O}_{\text{ForgeMsg}}$ is always 0 before index j .

We now argue that if $\mathcal{B}$ guesses the correct value of j , it also correctly computes $Z = e(g_1, g_2)^{yz}$ in the BDH experiment. Observe that $\mathcal{A}$ cannot win using a value of a (c_2, r', ctx) from a call to $\mathcal{O}_{\text{SendMsg}}$: the abort condition added in $\mathbf{H}_2$ ensures that there is a unique m for each c_2 in T_{seen} , and the experiment only reaches the line where $\text{win} \leftarrow 1$ if $(m^*, \text{ctx}^*) \notin T_{\text{seen}}$. Thus no (c_2, ctx) from a call to $\mathcal{O}_{\text{SendMsg}}$ can be used to win in a call to $\mathcal{O}_{\text{ForgeMsg}}$. This means it must be that the value of $H(c_2, r', \text{ctx})$ for the winning query to $\mathcal{O}_{\text{ForgeMsg}}$ must have been set to some Y_1^α , and for any winning report, $\mathcal{B}$ will know α such that $H(c_2, r', \text{ctx}) = Y_1^\alpha$.

The last condition for Moderate to output ctx instead of $\perp$ is

$$\sigma' = e(H(c_2, r', \text{ctx})^{k_m}, g_2),$$

Which by the bilinearity of the pairing is equivalent to

$$\sigma' = e(H(c_2, r', \text{ctx}), g_2)^{k_m}.$$

But since we have that $H(c_2, r', \text{ctx}) = Y_1^\alpha$ and we have assigned our values such that $k_m = z$, this is in turn equivalent to $\sigma' = e(Y_1^\alpha, g_2)^z$. Thus we have

$$\begin{aligned} \sigma' &= e(g_1^{y\alpha}, g_2)^z \\ \sigma' &= e(g_1, g_2)^{yz\alpha} \\ \sigma'^{1/\alpha} &= e(g_1, g_2)^{yz} = Z. \end{aligned}$$

$\square$

Theorem E.5 (Mod-Priv-2 Deniability). *Assuming that DDH is hard in $\mathbb{G}$, and modeling H as a random oracle, our third-party franking scheme Π_{MP2} (Mod-Priv-2), described in Construction 6.3, satisfies deniability (Definition B.3).*

Proof (sketch). Define the forgery algorithms as follows. Note that the only information the adversary receives in the deniability experiments is report_b , so it suffices to show that $\{\text{report}_0\}$ is distributed computationally indistinguishably from $\{\text{report}_1\}$. In both experiments, r and r' are selected uniformly at random, ctx is selected by the adversary, c_2 is a deterministic function of r and the adversary's input m , and c'_3 is an encryption under the moderator's pk_2 of r' . So the only piece computed differently across the two experiments is σ' .

Forge(pks, target, $k_p, k_{\text{reg}}, \text{ks}, \text{sk}_{m, \text{target}}, \text{sk}_{m, p, \text{target}}, m, \text{ctx}$)

```

pk1, pk2, k1→2, pkproc ← pks[target]
•, km ← skm, target
r ←R  $\mathcal{R}$ 
c2 ← Com.Commit(m, r)
r' ←R  $\mathbb{G}$ 
σ' ←R  $\mathbb{G}_T$ 
σ' ← e(H(c2, r', ctx)kp, pkproc)
σ' ← e(H(c2, r', ctx)km, g2)
c'3 ← RE.Enc(pk2, r')
report ← (r, c2, ctx, σ', c'3)
return report

```

Universal Deniability. The proof that σ' is indistinguishable between report_0 and report_1 first uses the hardness of DDH in G to replace c'_3 with an encryption of a new random value r'' and then invokes the fact that H is modeled as a random oracle to demonstrate that an adversary cannot distinguish between a real σ' and a random one in report_1 unless it guesses the value of r' , which occurs with at most negligible probability. We do not formalize this argument here because it is very similar to the proof of subpoena safety for this scheme, and the proof is presented formally there. The main difference is that subpoena safety deals with σ and c_3 , whereas universal deniability deals with σ' and c'_3 . But since only public values are needed to go from σ to σ' and c_3 to c'_3 , the reasoning about the former two transfers directly to a proof about the latter two.

Platform Compromise Deniability. The values of σ' in report_0 and report_1 are computed via the exact same operations, so they are distributed identically.

Moderator Compromise Deniability. In experiment $\text{MC-DENY}[\mathcal{A}, \Pi_{\text{MP2}}, \text{Forge}, N, 0]$, σ' is computed as follows.

$$\begin{aligned}
\sigma' &= e(\sigma, \text{pk}_{\text{proc}}) \\
&= e(H(c_2, r', \text{ctx})^{k_p}, k_{\text{reg}}^{k_m}) \\
&= e(H(c_2, r', \text{ctx})^{k_p}, g_2^{k_m/k_p}) \\
&= e(H(c_2, r', \text{ctx}), g_2)^{(k_m/k_p) \cdot k_p} \\
&= e(H(c_2, r', \text{ctx}), g_2)^{k_m} \\
&= e(H(c_2, r', \text{ctx})^{k_m}, g_2)
\end{aligned}$$

Since this is the same quantity computed in the Forge algorithm, σ' is distributed identically across both experiments.

Since in all three deniability experiments, report_0 is distributed indistinguishably from report_1 , Mod-Priv-2 satisfies deniability. $\square$

Theorem E.6 (Mod-Priv-2 Subpoena Safety). *Assuming that DDH is hard in $\mathbb{G}$, and modeling H as a random oracle, our third-party franking scheme Π_{MP2} (Mod-Priv-2), described in Construction 6.3, satisfies subpoena safety (Definition B.4).*

Specifically, for all efficient adversaries $\mathcal{A}$ who make at most Q queries to the $\mathcal{O}_{\text{SendMsg}}$ oracle and Q_{RO} queries to the random oracle H , all $N, Q, Q_{\text{RO}} \in \mathbb{N}$, and for any security parameter λ , there exists a DDH adversary $\mathcal{B}$ for $\mathbb{G}$ such that

$$\begin{aligned}
&\text{SUBPOENAAAdv}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q, \lambda] \\
&\leq 2 \cdot \text{DDHAdv}[\mathcal{B}, \mathbb{G}] + 2QQ_{\text{RO}}/q.
\end{aligned}$$

Proof. The proof of this theorem is very similar to that of Theorem D.6, with the only changes dealing with how the tag σ is computed. Define the forgery algorithm as follows.

Forge(pks, target, k, m, ctx)

```

pk1, pk2, k1→2, pkproc ← pks[target]
s, epk ← RE.MaskKey(pk1)
ke→2 ← RE.ReKeyGen(s, k1→2)
r, r' ←R  $\mathcal{R} \times \mathbb{G}$ 
c1 ← Enc(k, (m, target, r, ke→2))
c2 ← Com.Commit(m, r)
c3 ← RE.Enc(epk, r')
st ← (c3, epk, ctx)
σ ←R  $\mathbb{G}_1$ 
return (c1, c2, σ, st)

```

Observe that the only difference between the output received_1 of Forge and the values of $(c_1, c_2, \sigma, \text{st})$ output in received_0 is the computation of σ . We will show that these values are computationally indistinguishable through a series of hybrids.

- $\mathbf{H}_0$: This is the subpoena safety experiment $\text{SUBPOENA}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q, 0, \lambda]$.
- $\mathbf{H}_1$: This hybrid is identical to the preceding one, except we replace the value pk^r in RE.Enc with a uniformly random element of $\mathbb{G}$. This hybrid is indistinguishable from the preceding one by the hardness of DDH in $\mathbb{G}$. We omit the proof, which is identical to the proof of Lemma D.7.
- $\mathbf{H}_2$: This hybrid is identical to the preceding one, except we replace the value r' encrypted in c_3 with a uniformly random r'' . This hybrid is distributed identically to the preceding one because the value r'' is multiplied by g^r for an independently random r not used elsewhere (this variable name is overloaded in RE and is distinct from the r used in Send).
- $\mathbf{H}_3$: This hybrid is identical to the preceding one, except the experiment aborts and outputs 0 if the adversary ever queries the random oracle H on input $(\cdot, r', \cdot)$ for any value of r' sampled during a call to $\mathcal{O}_{\text{SendMsg}}$. This hybrid is indistinguishable from the previous one as long as the new abort condition is not triggered. But since r' appears nowhere in the view of the adversary except as an input to H , the probability that the adversary makes a query that triggers the abort condition is $1/q$ for each call to H for each value of r' , since

r' is chosen uniformly at random from the group $\mathbb{G}$ of order q . Each call to $\mathcal{O}_{\text{SendMsg}}$ selects another random r' , so by the union bound, the overall probability of triggering the abort condition is at most QQ_{RO}/q .

- **H₄** : This hybrid is identical to the preceding one, except we compute the tags σ as $\sigma \xleftarrow{\mathbb{R}} \mathbb{G}_1$. This hybrid is distributed identically to the preceding one where $\sigma = H(c_2, r', \text{ctx})^{k_p}$. Since $H(\cdot, r', \cdot)$ appears uniformly random to an adversary who does not trigger the abort condition added in **H₃**, the value of σ is a uniformly random output from the random oracle raised to k_p , which will still be random.
- **H₅** : This hybrid undoes the change made in **H₃**, removing the abort condition added in that hybrid. It is statistically indistinguishable from the preceding hybrid by the same reasoning provided above, so an adversary can distinguish with probability at most QQ_{RO}/q .
- **H₆** : This hybrid undoes the change made in **H₂**, encrypting r' in c_3 instead of a new random r'' . This hybrid is distributed identically to the preceding one by the same reasoning provided above.
- **H₇** : This hybrid undoes the change made in **H₁**, using the value of pk^r in RE.Enc as specified in the original scheme. This hybrid is indistinguishable from the preceding one by the hardness of DDH in $\mathbb{G}$. We omit the proof, which is identical to the proof of Lemma D.7.

Observe that hybrid **H₇** is identical to the subpoena safety experiment $\text{Pr}[\text{SUBPOENA}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q, 1, \lambda]]$. The proof of the theorem thus follows from the indistinguishability of each successive pair of hybrids and the triangle inequality. $\square$

Theorem E.7 (Mod-Priv-2 Moderator Privacy). *Assuming that the underlying symmetric encryption scheme is CPA secure, our third-party franking scheme Π_{MP2} (Mod-Priv-2), described in Construction 6.3, satisfies moderator privacy (Definition B.5).*

Specifically, for any adversary $\mathcal{A}$ attacking the moderator privacy of our scheme Π_{MP2} , for all $N, U \in \mathbb{N}$, and for any security parameter λ , there exists an adversary $\mathcal{B}$ attacking the CPA security of Enc , such that

$$\text{MODPRIVAdv}[\mathcal{A}, \Pi_{\text{MP2}}, N, U, \lambda] \leq 2U \cdot \text{CPAAAdv}[\mathcal{B}, \lambda].$$

We omit a proof of this theorem as the proof is identical to that of Theorem D.9.

Theorem E.8 (Mod-Priv-2 Moderator Binding). *Assuming that DDH is hard in $\mathbb{G}$ and modeling H as a random oracle, our third-party franking scheme Π_{MP2} (Mod-Priv-2), described in Construction 6.3, satisfies moderator binding (Definition B.6).*

Specifically, for any adversary $\mathcal{A}$ attacking the moderator binding of our scheme Π_{MP2} who makes at most Q queries to the oracle $\mathcal{O}_{\text{Mod}}$ and Q_{RO} queries to the random oracle H , for all $N, Q, Q_{\text{RO}} \in \mathbb{N}$, and for any security parameter λ , there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ attacking DDH

in $\mathbb{G}$ and BDH for the groups in the pairing e , respectively, such that

$$\begin{aligned} \text{MODBINDAdv}[\mathcal{A}, \Pi_{\text{MP2}}, N, Q, \lambda] \\ \leq \text{DDHAdv}[\mathcal{B}, \mathbb{G}] + Q \cdot \text{BDHAdv}[\mathcal{C}, e] \\ + QQ_{\text{RO}}/q + Q^2/q. \end{aligned}$$

Proof. We refer to the values of σ_{target} , r' , and i present during the j th call to $\mathcal{O}_{\text{Mod}}$ as $\sigma_{\text{target},j}$, r'_j , and i_j , respectively. The proof begins with a series of hybrids.

- **H₀**: This is the moderator binding experiment $\text{MODBINDAdv}(\mathcal{A}, \Pi_{\text{MP2}}, N, Q, \lambda)$.
- **H₁**: This hybrid is identical to the preceding one, except in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$, we replace the value pk^r in RE.Enc with a uniformly random element of $\mathbb{G}$. This hybrid is indistinguishable from the preceding one by the hardness of DDH in $\mathbb{G}$. We omit a full proof, as it is identical to that of Lemma D.11. Observe that in this hybrid, the values of r' chosen in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$ are no longer a part of the adversary's view, as they are masked with a random value g^r in the ciphertext c_3 , for r used nowhere else.
- **H₂**: This hybrid adds an abort condition to the experiment. The experiment aborts if the adversary ever produces a value of c'_3 such that $\text{RE.Dec}(\text{sk}_{m,\text{target}}, c'_3) = r'$ for any value of r' sampled in a call to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$. This hybrid is statistically indistinguishable from the preceding one. Since r' never appears in the adversary's view in calls to $\mathcal{O}_{\text{Mod}}$ where $i \neq \text{target}$, the probability of this abort condition being triggered is $1/q$ times the number of values of c'_3 produced by the experiment and the number of values of r' sampled during the experiment when $i \neq \text{target}$. These values increase by at most one per call to $\mathcal{O}_{\text{Mod}}$, so the probability of the abort condition being triggered is at most Q^2/q .
- **H₃** : This hybrid adds an additional abort condition to the experiment. The experiment aborts if the adversary ever queries H at an input $(\cdot, r', \cdot)$ that is later selected as the randomness used in a call to Process in the $\mathcal{O}_{\text{Mod}}$ oracle.

This hybrid is statistically indistinguishable from the preceding one. Since the values of r' in Process are chosen uniformly at random from $\mathbb{G}$, choices of r' made after an adversary's query to H are independent of the adversary's query. Thus for each query the adversary makes to H , the probability that the value of r' used in that query later appears in a call to Process is Q/q . Taking a union bound over the number of queries made to H yields a probability of QQ_{RO}/q as an upper bound on the probability of an adversary triggering this abort condition.

We show that for any adversary $\mathcal{A}$ who wins the MODBIND experiment in **H₃**, we can build an adversary $\mathcal{B}$ who uses $\mathcal{A}$ to win the BDH experiment with probability $1/Q$, thus completing the proof of the theorem. This proof is similar to the proof of Lemma E.4.

Adversary $\mathcal{B}$ plays the role of the BDH adversary while simulating the $\mathbf{H}_3$ challenger to $\mathcal{A}$. Recall that in our variant of BDH, the adversary $\mathcal{B}$ receives $g_1, g_2, X_1 = g_1^x, X_2 = g_2^{1/x}, Y_1 = g_1^y, Y_2 = g_2^{z/x}$ for $x, y, z \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and is tasked with computing a value $Z = e(g_1, g_2)^{yz}$.

In our reduction, $\mathcal{B}$ sets $k_{\text{reg}} \leftarrow X_2$ and $\text{pk}_{\text{proc}, \text{target}} \leftarrow Y_2$. This has the effect of implicitly setting $k_p = x$ and $k_m = z$ for the target moderator. When computing the value of σ in calls to $\mathcal{O}_{\text{Mod}}$, adversary $\mathcal{B}$ samples a random $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and sets $\sigma \leftarrow X_1^\alpha$. This implicitly sets $H(c_2, r', \text{ctx})^{k_p} = g_1^{x\alpha}$ and $H(c_2, r', \text{ctx}) = g_1^\alpha$, which is what $\mathcal{B}$ sends to $\mathcal{A}$ if it queries the random oracle at (c_2, r', ctx) later in the experiment.

When the adversary $\mathcal{A}$ makes calls to the random oracle for a fresh input (c_2, r', ctx) outside of $\mathcal{O}_{\text{Mod}}$, $\mathcal{B}$ samples a fresh random $\alpha \xleftarrow{\mathbb{R}} \mathbb{Z}_p$ and returns Y_1^α . The abort condition added in $\mathbf{H}_3$ ensures that an output to H assigned in this way never needs to be reassigned in a call to Process in $\mathcal{O}_{\text{Mod}}$.

The adversary $\mathcal{B}$ selects a query index $j \xleftarrow{\mathbb{R}} \{0, \dots, Q-1\}$ as its guess of the first call to $\mathcal{O}_{\text{Mod}}$ that results in reaching the line $\text{win} \leftarrow 1$. For queries before j where $i = \text{target}$, it always outputs $\text{res} = \perp$. On query j , adversary $\mathcal{B}$ submits $\sigma'^{1/\alpha}$ as its guess of Z in the BDH experiment, where σ' is the value submitted by $\mathcal{A}$ in report and α is the exponent chosen to make $H(c_2, r', \text{ctx}) \leftarrow Y_1^\alpha$ for the values of (c_2, r', ctx) in $\mathcal{A}$'s report. We will show below that $\mathcal{B}$ will always know this α .

Observe that if $\mathcal{B}$ guesses the correct value of j , it provides $\mathcal{A}$ with a perfect simulation of the $\mathbf{H}_3$ security experiment where $k_p = x$, $k_m = z$, and each evaluation of H appears independently random. Since $\mathcal{B}$ has guessed correctly, the output win of $\mathcal{O}_{\text{Mod}}$ is always 0 before index j .

We now argue that if $\mathcal{B}$ guesses the correct value of j , it also correctly computes $Z = e(g_1, g_2)^{yz}$ in the BDH experiment. Observe that the abort conditions in the experiment ensure that $\mathcal{A}$ cannot win using a value of (c_2, r', ctx) from a call to $\mathcal{O}_{\text{Mod}}$. For the case where $\mathcal{O}_{\text{Mod}}$ is called with $i \neq \text{target}$, this is enforced by the abort condition in $\mathbf{H}_3$, which ensures that no r' from such a call ever appears in $\mathcal{A}$'s report. In the case where $i = \text{target}$, observe that (1) it is impossible that the choice of moderator in query j is the target, because it is not possible to win on a query where $i = \text{target}$, and (2) the value of ctx from every call to $\mathcal{O}_{\text{Mod}}$ where $i = \text{target}$ will be in T_{seen} . So reusing ctx cannot lead to $\text{win} \leftarrow 1$ because the oracle will return early with $\perp$ in that case. This means it must be that the value of $H(c_2, r', \text{ctx})$ for the winning query to $\mathcal{O}_{\text{Mod}}$ must have been set to Y_1^α , and for any winning report, $\mathcal{B}$ will know α such that $H(c_2, r', \text{ctx}) = Y_1^\alpha$.

The last condition for Moderate to output ctx instead of $\perp$ is

$$\sigma' = e(H(c_2, r', \text{ctx})^{k_m}, g_2),$$

Which by the bilinearity of the pairing is equivalent to

$$\sigma' = e(H(c_2, r', \text{ctx}), g_2)^{k_m}.$$

But since we have that $H(c_2, r', \text{ctx}) = Y_1^\alpha$ and we have assigned our values such that $k_m = z$, this is in turn equivalent to $\sigma' = e(Y_1^\alpha, g_2)^z$. Thus we have

$$\begin{aligned} \sigma' &= e(g_1^{y\alpha}, g_2)^z \\ \sigma' &= e(g_1, g_2)^{yz\alpha} \\ \sigma'^{1/\alpha} &= e(g_1, g_2)^{yz} = Z. \end{aligned}$$

□